

Natural Language Generation

Antoine Bosselut

Announcements

- **Midterm:** April 15th confirmed!

Midterm Logistics

- **Date:**

- Wednesday, April 15th, 2026
- 11:30 - 13:00

- **Location:** 6 rooms on campus

- AAC 0 20, AAC 1 14, AAC 1 08, CO 3, ELA 1, MA B1 11
- Each student will be pre-assigned to one of the rooms.
 - **YOUR EXAM COPY WILL BE IN THE ROOM WHERE YOU WERE ASSIGNED.**
 - **YOU MUST GO TO THE CORRECT ROOM.**
 - **NO EXTRA EXAMS WILL BE AVAILABLE IN OTHER ROOMS.**

- **Schedule**

- **A few days before:** room assignments and seating chart released on Moodle
- **11:00** on the day of: Exam rooms set up
- **11:20** on the day of: Exam hall doors open and students take their seat.
- **11:30** on the day of: Exam begins
- **During exam:** EPFL ID Cards checked (**don't forget CAMIPRO!**)
- **13:00** on the day of: Exams collected

Midterm Format

- 30 Multiple Choice QA
 - From **lectures** AND **exercise sessions** (up to and including **Week 7**)
- Materials
 - 1 double-sided A4 crib sheet allowed
 - Non-programmable calculators allowed
 - We will check and you will lose time (and everything on calculator) performing memory wipe
 - CAMIPRO card
- No phones or other electronic devices allowed

Today's Outline

- **Lecture:**

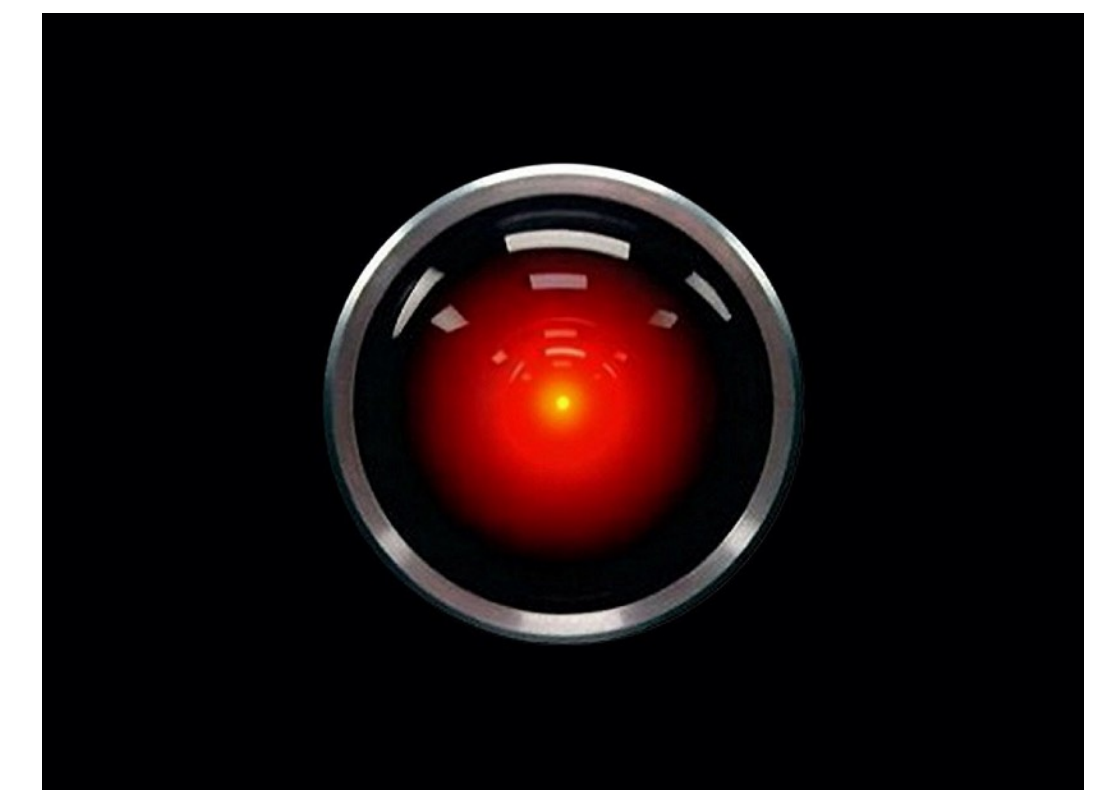
- **Text Generation:** Task
- **Greedy Decoding:** Argmax, Beam Search
- **Sampling Methods:** Top-k, Top-p
- **Training Challenges:** Exposure bias, Reinforcement Learning

- **Tomorrow:**

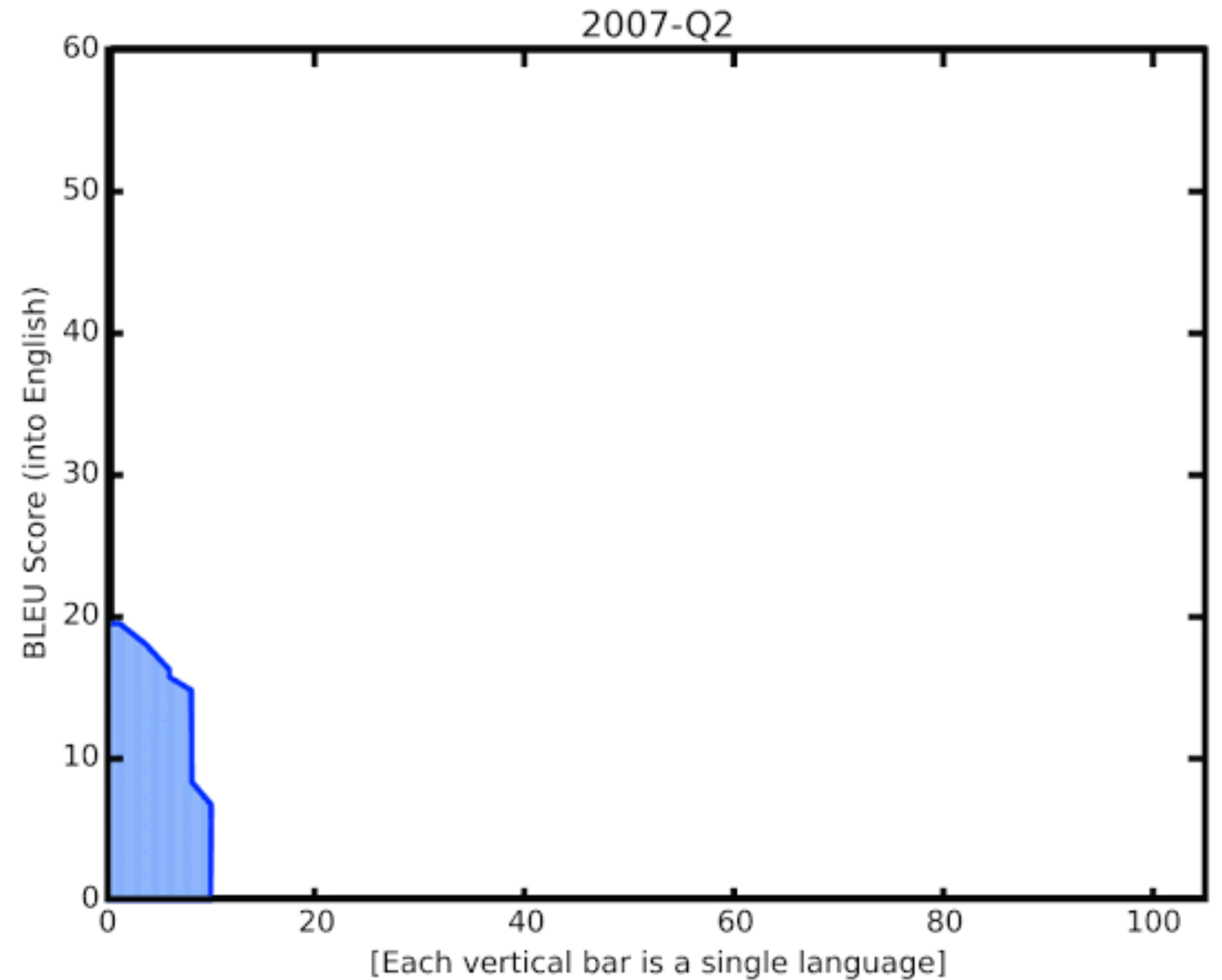
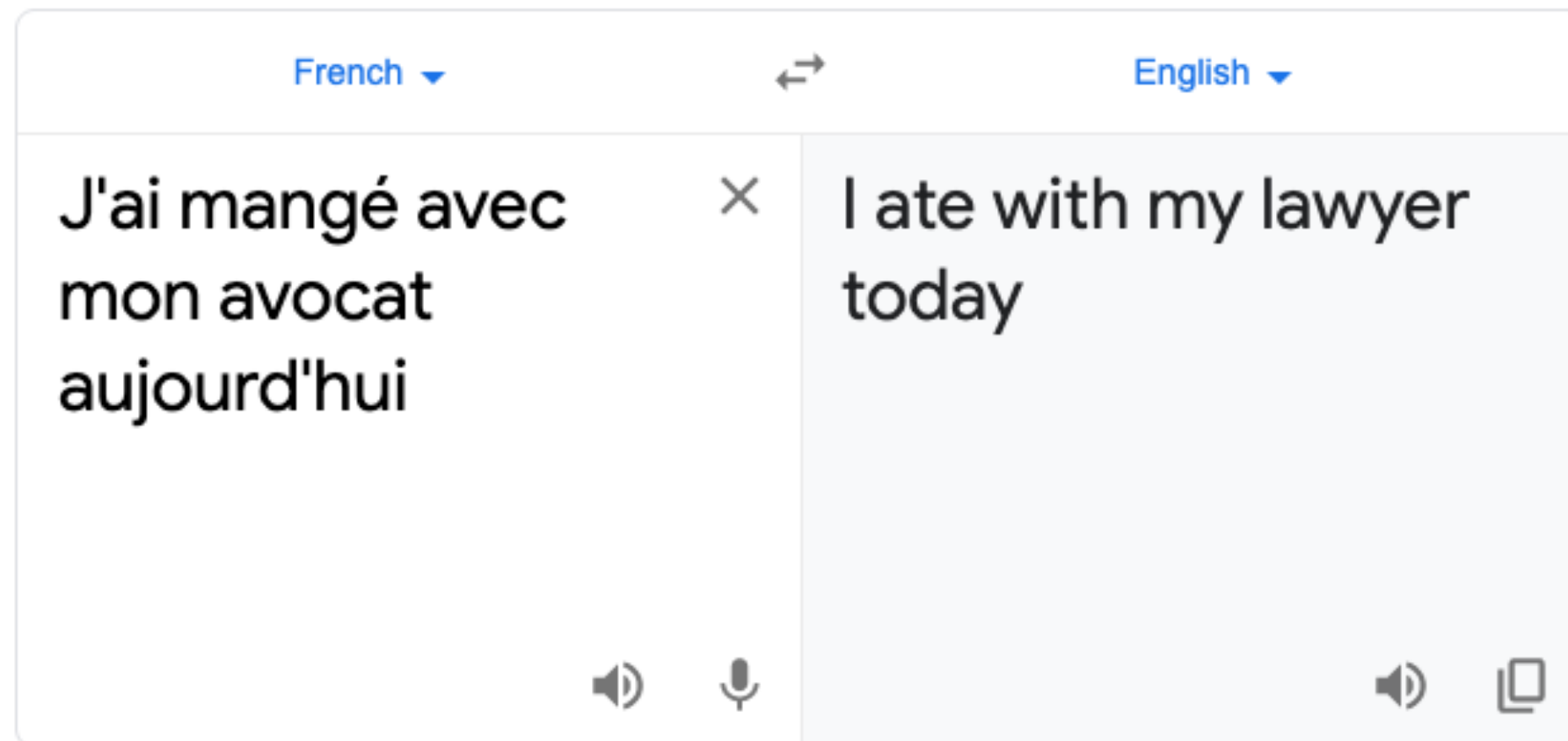
- **Lecture:** Text generation evaluation
- **Review of Week 5 Exercise Session:** Dataset Biases & Prompting

What is natural language generation?

- Natural language generation (NLG) is a sub-field of natural language processing
- Focused on building systems that automatically produce **coherent** and **useful** written or spoken text for human consumption
- NLG systems are already changing the world we live in...

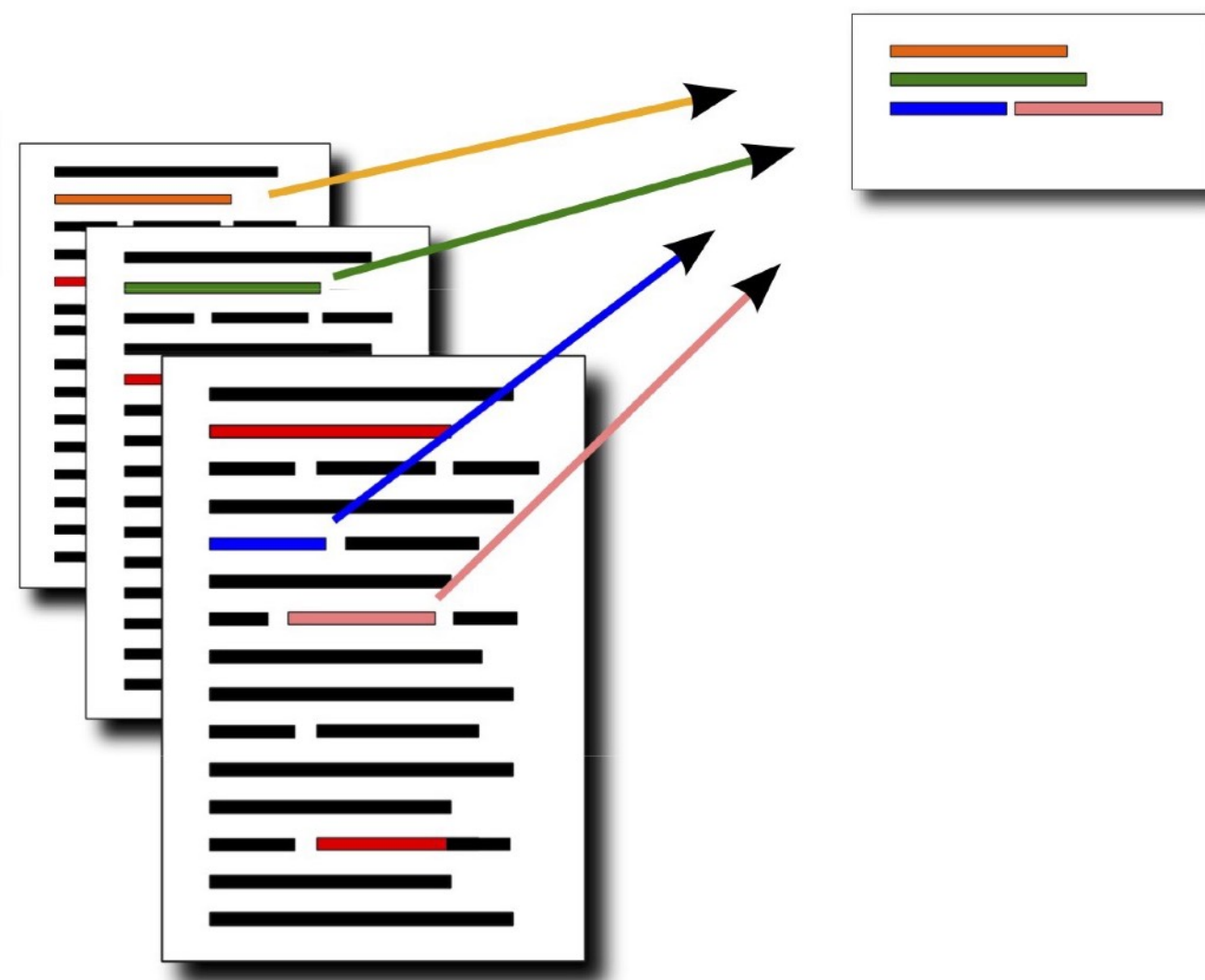


Machine Translation



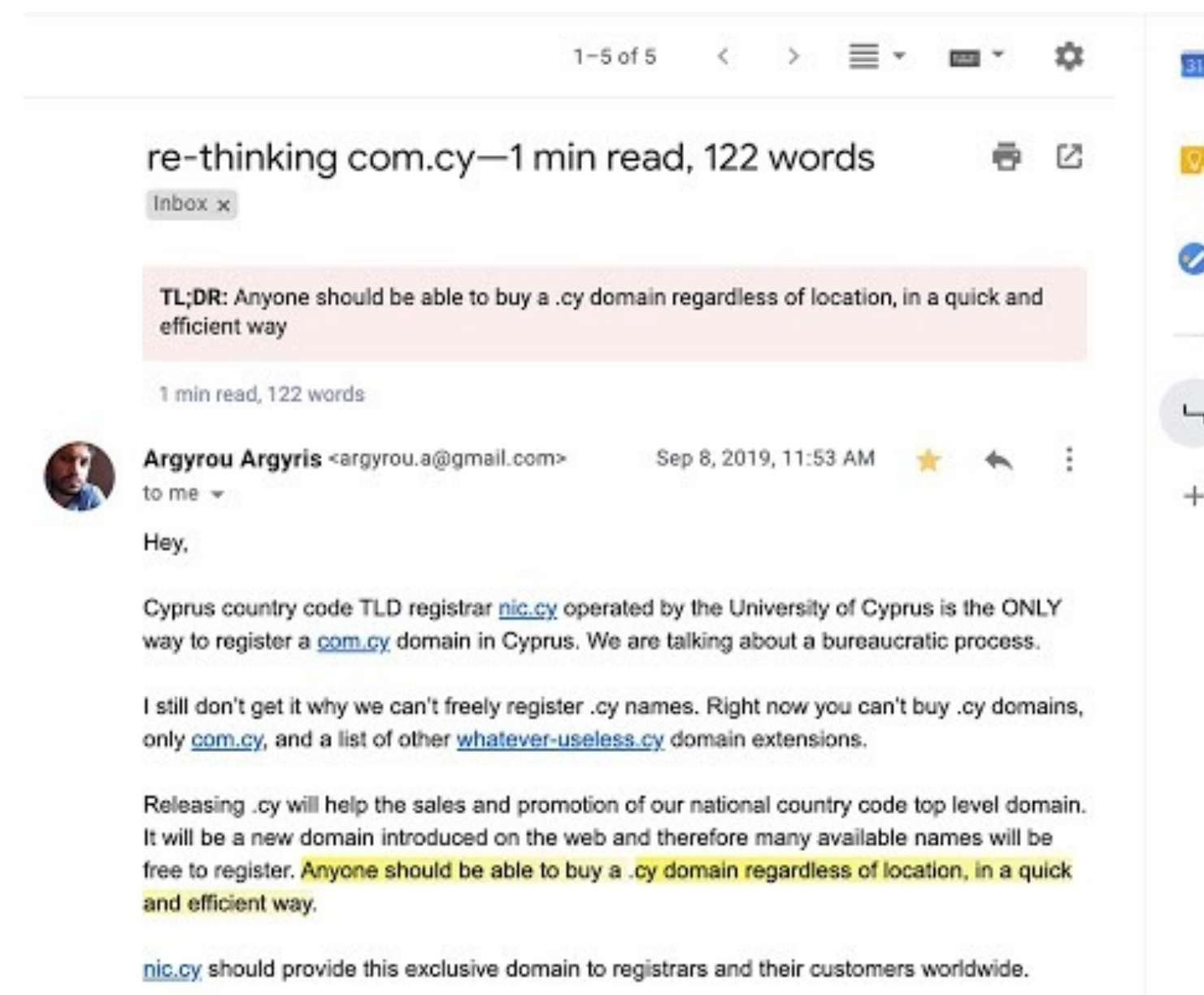
Summarization

Document Summarization



<http://mogren.one/lic/>

E-mail Summarization



<https://chrome.google.com/webstore/detail/gmail-summarization/>

Meeting Summarization

C: Looking at what we've got, we we want an LCD display with a spinning wheel.
B: You have to have some push-buttons, don't you?
C: Just spinning and not scrolling, I would say.
B: I think the spinning wheel is definitely very now.
A: but since LCDs seems to be uh a definite yes,
C: We're having push-buttons on the outside
C: and then on the inside an LCD with spinning wheel,

Decision Abstract (Summary):

The remote will have push buttons outside, and an LCD and spinning wheel inside.

A: and um I'm not sure about the buttons being in the shape of fruit though.
D: Maybe make it like fruity colours or something.
C: The power button could be like a big apple or something.
D: Um like I'm just thinking bright colours.

Problem Abstract (Summary):

How to incorporate a fruit and vegetable theme into the remote.

(Wang and Cardie, ACL 2013)

Data-to-Text Generation

Table Title: Robert Craig (American football)
Section Title: National Football League statistics
Table Description:None

RUSHING							RECEIVING				
YEAR	TEAM	ATT	YDS	AVG	LNG	TD	NO.	YDS	AVG	LNG	TD
1983	SF	176	725	4.1	71	8	48	427	8.9	23	4
1984	SF	155	649	4.2	28	4	71	675	9.5	64	3
1985	SF	214	1050	4.9	62	9	92	1016	11	73	6
1986	SF	204	830	4.1	25	7	81	624	7.7	48	0
1987	SF	215	815	3.8	25	3	66	492	7.5	35	1
1988	SF	310	1502	4.8	46	9	76	534	7.0	22	1
1989	SF	271	1054	3.9	27	6	49	473	9.7	44	1
1990	SF	141	439	3.1	26	1	25	201	8.0	31	0
1991	RAI	162	590	3.6	15	1	17	136	8.0	20	0
1992	MIN	105	416	4.0	21	4	22	164	7.5	22	0
1993	MIN	38	119	3.1	11	1	19	169	8.9	31	1
Totals	-	1991	8189	4.1	71	56	566	4911	8.7	73	17

Target Text: Craig finished his eleven NFL seasons with 8,189 rushing yards and 566 receptions for 4,911 receiving yards.

(Parikh et al., EMNLP 2020)

TEAM	WIN	LOSS	PTS	FG_PCT	RB	AS ...
Heat	11	12	103	49	47	27
Hawks	7	15	95	43	33	20

PLAYER	AS	RB	PT	FG	FGA	CITY ...
Tyler Johnson	5	2	27	8	16	Miami
Dwight Howard	4	17	23	9	11	Atlanta
Paul Millsap	2	9	21	8	12	Atlanta
Goran Dragic	4	2	21	8	17	Miami
Wayne Ellington	2	3	19	7	15	Miami
Dennis Schroder	7	4	17	8	15	Atlanta
Rodney McGruder	5	5	11	3	8	Miami
Thabo Sefolosha	5	5	10	5	11	Atlanta
Kyle Korver	5	3	9	3	9	Atlanta
...						

The Atlanta Hawks defeated the Miami Heat , 103 - 95 , at Philips Arena on Wednesday . Atlanta was in desperate need of a win and they were able to take care of a shorthanded Miami team here . Defense was key for the Hawks , as they held the Heat to 42 percent shooting and forced them to commit 16 turnovers . Atlanta also dominated in the paint , winning the rebounding battle , 47 - 34 , and outscoring them in the paint 58 - 26.The Hawks shot 49 percent from the field and assisted on 27 of their 43 made baskets . This was a near wire - to - wire win for the Hawks , as Miami held just one lead in the first five minutes . Miami (7 - 15) are as beat - up as anyone right now and it 's taking a toll on the heavily used starters . Hassan Whiteside really struggled in this game , as he amassed eight points , 12 rebounds and one blocks on 4 - of - 12 shooting ...

(Wiseman and Rush., EMNLP 2017)

MR:

name[The Eagle],
eatType[coffee shop],
food[French],
priceRange[moderate],
customerRating[3/5],
area[riverside],
kidsFriendly[yes],
near[Burger King]

NL:

"The three star coffee shop, The Eagle, gives families a mid-priced dining experience featuring a variety of wines and cheeses. Find The Eagle near Burger King."

(Dusek et. al., INLG 2019)

Visual Description Generation



bowls are food in triangular shape are sitting on table
table filled with many plates of various breakfast foods
table topped with lots of different types of donuts



hotdog stand on busy street
man in white t shirt is holding umbrella and ice cream cart
man in white shirt is pushing his cart down street



man in graduation robes riding bicycle
cyclist giving thumbs up poses with his bicycle by right
of way sign at park
man riding motorcycle on street



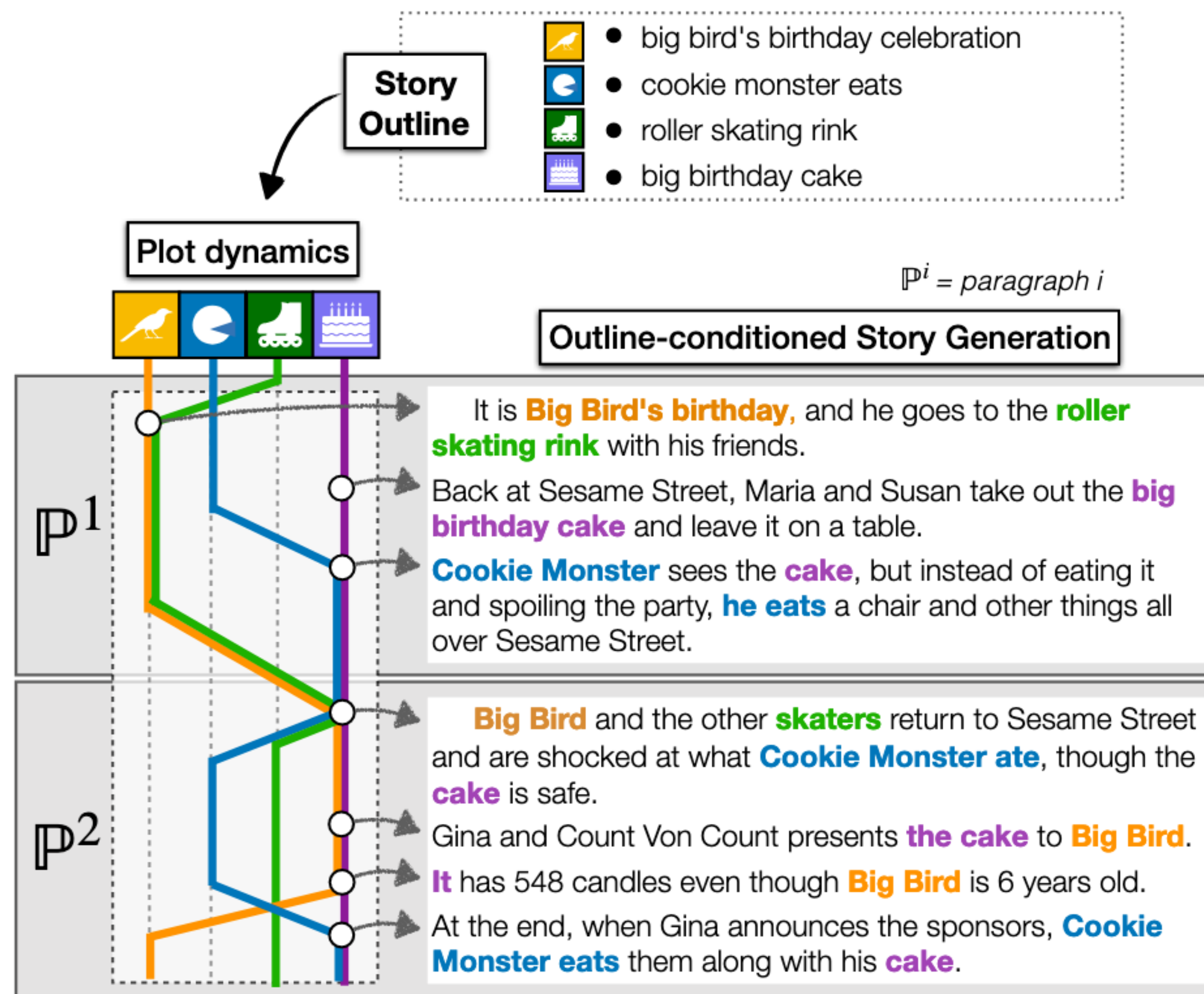
one man and two women sitting in living room
man and woman are playing wii game while woman
sits on couch with wine glass in her hand
group of people sitting on couch with their laptops



Two children are sitting at a table in a restaurant. The children are one little girl and one little boy. The little girl is eating a pink frosted donut with white icing lines on top of it. The girl has blonde hair and is wearing a green jacket with a black long sleeve shirt underneath. The little boy is wearing a black zip up jacket and is holding his finger to his lip but is not eating. A metal napkin dispenser is in between them at the table. The wall next to them is white brick. Two adults are on the other side of the short white brick wall. The room has white circular lights on the ceiling and a large window in the front of the restaurant. It is daylight outside.

Creative Generation

Stories & Narratives



(Rashkin et al., EMNLP 2020)

Poetry

Vocabulary

Style

Encourage words: **Reset Style**

curse words: - 0 +
topical words: - 0 +
repetition: - 0 +
monosyllable words: - 0 +
alliteration: - 0 +
sentiment: - 0 +
word length: - 0 +
concrete words: - 0 +

love

Poem

☆☆☆☆☆

My lovely lady sweet and sweet *temptation*,
The lucky woman on the *wedding night*,
I really need a friend of *consolation*,
A lonely part of you and *me tonight*.

(a) Poem generated with default style settings

Vocabulary

Style

Encourage words: **Reset Style**

curse words: - 0 +
topical words: - 0 +
repetition: - 0 +
monosyllable words: - 0 +
alliteration: - 0 +
sentiment: - 0 +
word length: - 0 +
concrete words: - 0 +

love

Poem

☆☆☆☆☆ Thanks for your feedback !

My merry little love and sweet *temptation*,
The lucky lady on a *wedding night*,
She sings the sweetest song of *consolation*,
A lovely dream of you and *me tonight*.

(b) Poem generated with user adjusted style settings

(Ghazvininejad et al., ACL 2017)

All-in-one: ChatGPT

AN

Can you give me a two-day itinerary for a weekend trip to Paris ? Please don't include more than three activities per day.



Day 1:

1. Visit the Eiffel Tower - Start your day with the iconic symbol of Paris, the Eiffel Tower. Take a stroll around the Champ de Mars and enjoy the views from the tower.
2. Explore the Louvre Museum - After lunch, head to the Louvre Museum and admire the world-famous artwork such as the Mona Lisa and the Venus de Milo.
3. Stroll along the Seine River - In the evening, take a romantic stroll along the Seine River and enjoy the Parisian atmosphere.

Day 2:

1. Visit Notre-Dame Cathedral - Start your day by visiting one of the most famous Gothic cathedrals in the world, Notre-Dame Cathedral.
2. Explore the Marais neighborhood - After lunch, explore the Marais neighborhood, known for its historical architecture, trendy boutiques, and restaurants.
3. Enjoy a Cabaret Show - In the evening, enjoy a cabaret show at the famous Parisian venue, the Moulin Rouge, and end your trip with a memorable experience.

What is natural language generation?

Any task involving text production for human consumption requires natural language generation

What is natural language generation?

Any task involving text production for human consumption requires natural language generation

How do we make deep learning models generate natural language?

Basics of natural language generation

- Most text generation are autoregressive models — they predict **next tokens** based on the values of **past tokens**
- In autoregressive text generation models, at each time step t , our model takes in a sequence of tokens of text as input $\{y\}_{<t}$ and outputs a new token, $\hat{y}_t$

Basics of natural language generation

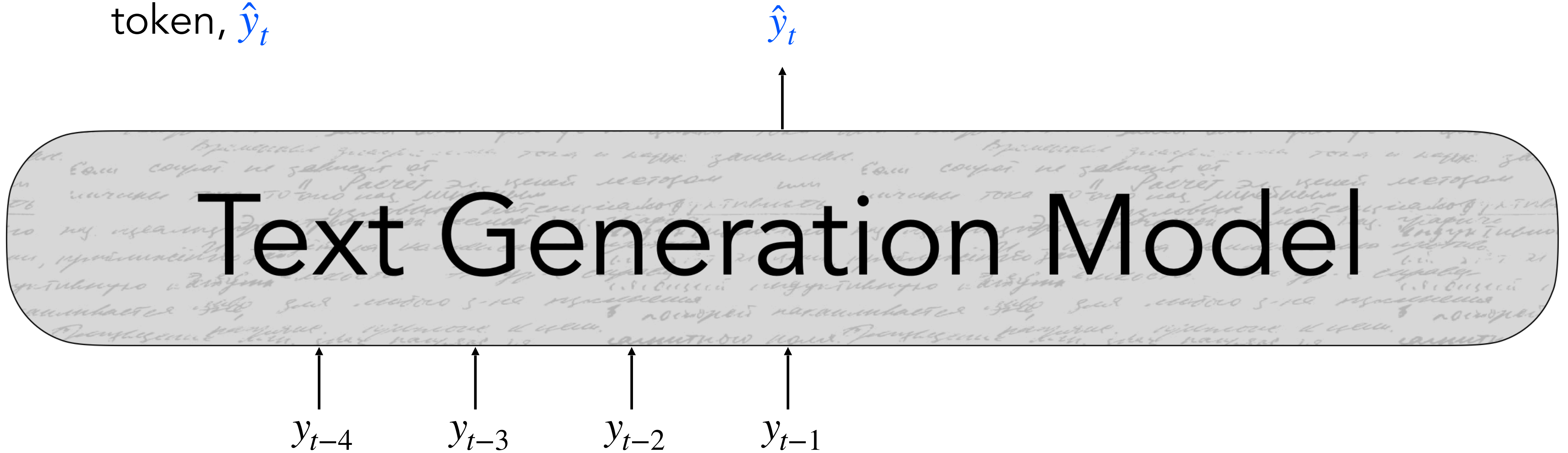
- In autoregressive text generation models, at each time step t , our model takes in a sequence of tokens of text as input $\{y\}_{<t}$ and outputs a new token, $\hat{y}_t$



Text Generation Model

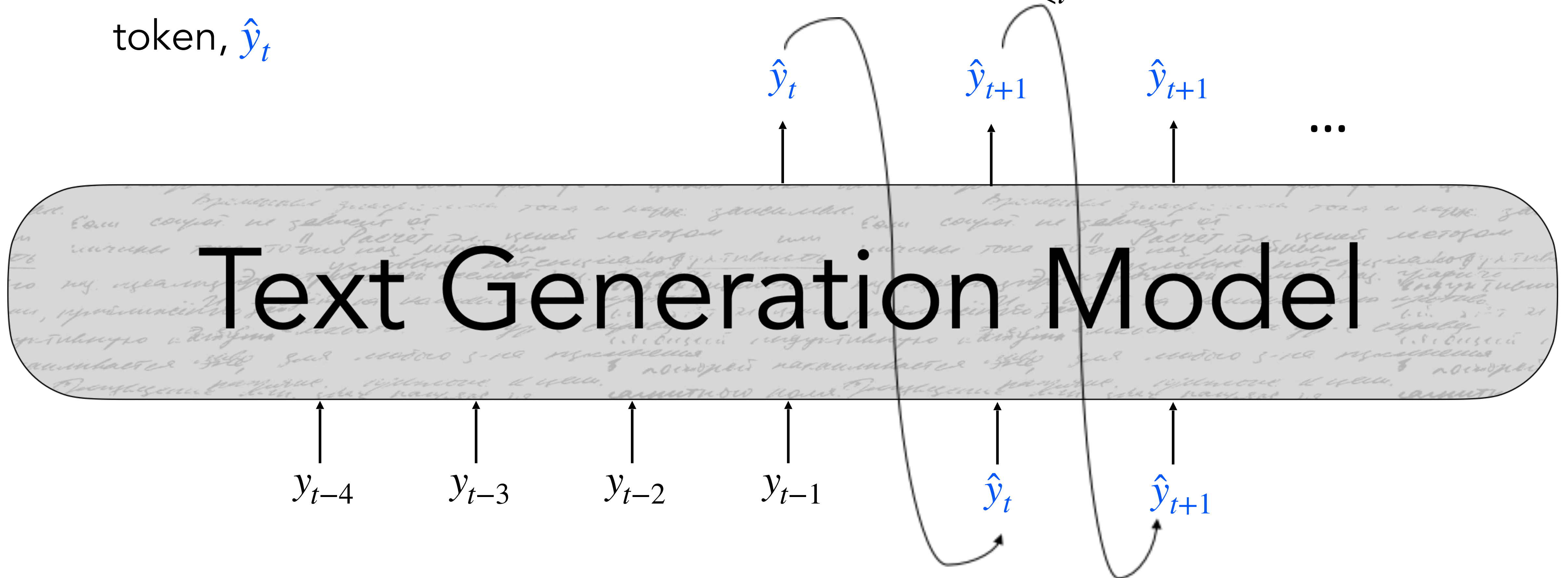
Basics of natural language generation

- In autoregressive text generation models, at each time step t , our model takes in a sequence of tokens of text as input $\{y\}_{<t}$ and outputs a new token, $\hat{y}_t$



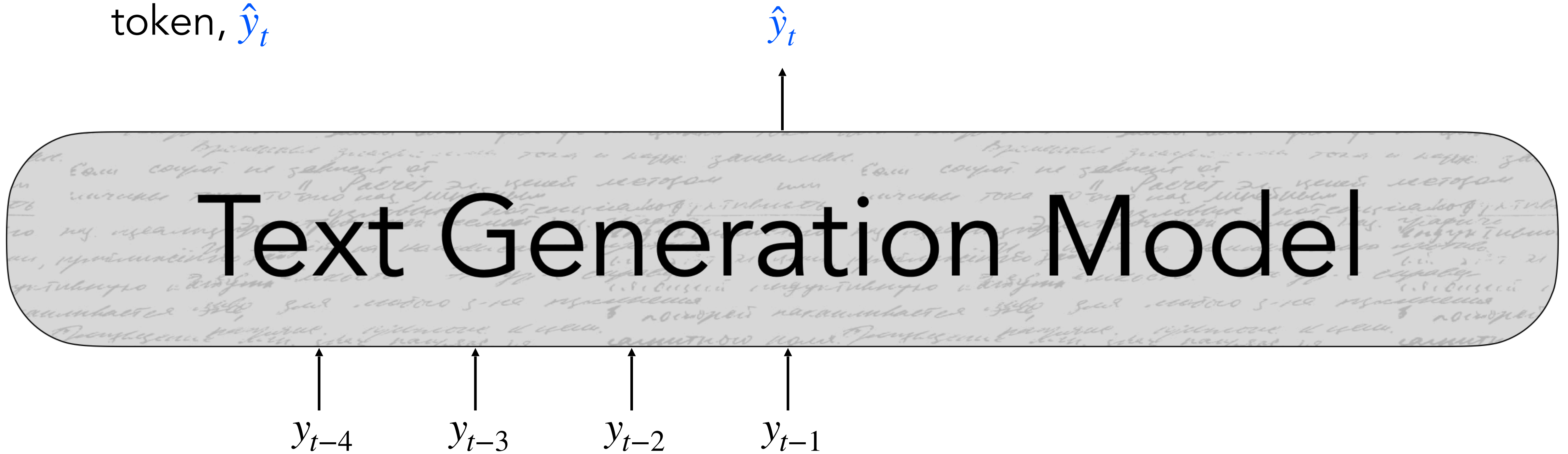
Basics of natural language generation

- In autoregressive text generation models, at each time step t , our model takes in a sequence of tokens of text as input $\{y\}_{<t}$ and outputs a new token, $\hat{y}_t$



A look at a single step

- In autoregressive text generation models, at each time step t , our model takes in a sequence of tokens of text as input $\{y\}_{<t}$ and outputs a new token, $\hat{y}_t$



A look at a single step

- At each time step t , our model computes a vector of scores for each token in our vocabulary, $\mathbf{S} \in \mathbb{R}^V$:

$$\mathbf{S} = f\left(\{y_{<t}\}, \theta\right)$$

$f(\cdot)$ is your model

- Then, we compute a probability distribution P over $w \in V$ using these scores:

$$P\left(y_t = w \mid \{y_{<t}\}\right) = \frac{\exp(\mathbf{S}_w)}{\sum_{w' \in V} \exp(\mathbf{S}_{w'})}$$

A look at a single step

- At each time step t , our model computes a vector of scores for each token in our vocabulary, $\mathbf{S} \in \mathbb{R}^V$:

$$\mathbf{S} = f\left(\{y_{<t}\}, \theta\right)$$

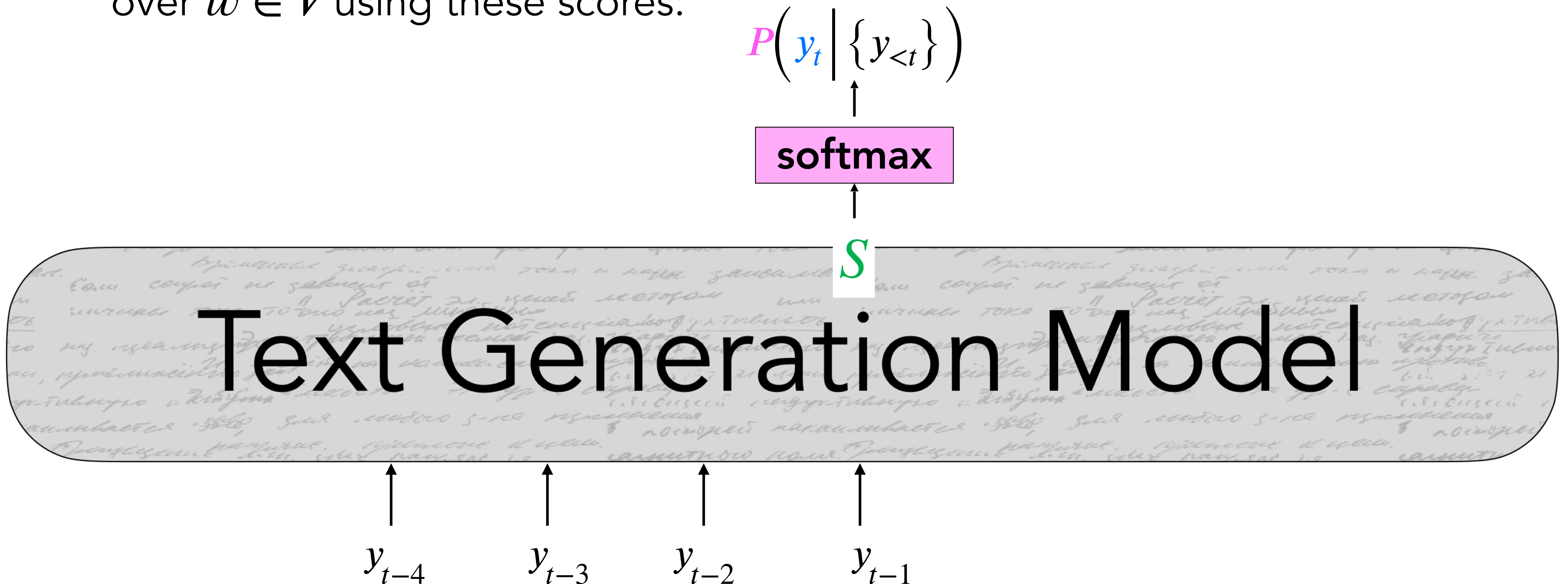
$f(\cdot)$ is your model

- Then, we compute a probability distribution P over $w \in V$ using these scores:

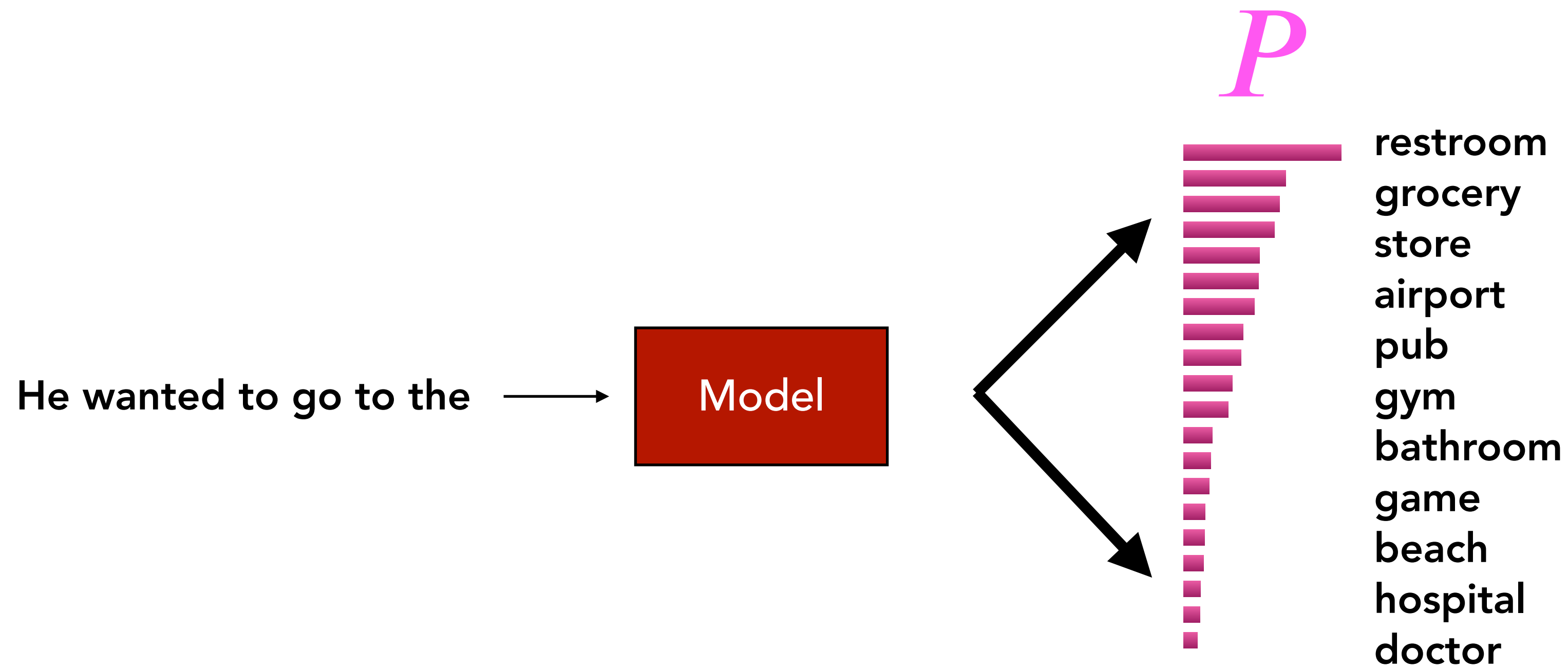
$$P\left(y_t \mid \{y_{<t}\}\right) = \frac{\exp(\mathbf{S}_w)}{\sum_{w' \in V} \exp(\mathbf{S}_{w'})}$$

A look at a single step

- At each time step t , our model computes a vector of scores for each token in our vocabulary, $\mathbf{S} \in \mathbb{R}^V$. Then, we compute a probability distribution P over $w \in V$ using these scores:



A look at a single step



- At inference time, our **decoding algorithm** defines a function to select a token from this distribution *P*:

$$\hat{y}_t = g(P(y_t | \{y_{<t}\}))$$

g(.) is your decoding algorithm

Basics: What are we trying to do?

- We train the model to minimize the negative loglikelihood of predicting the next token in the sequence:

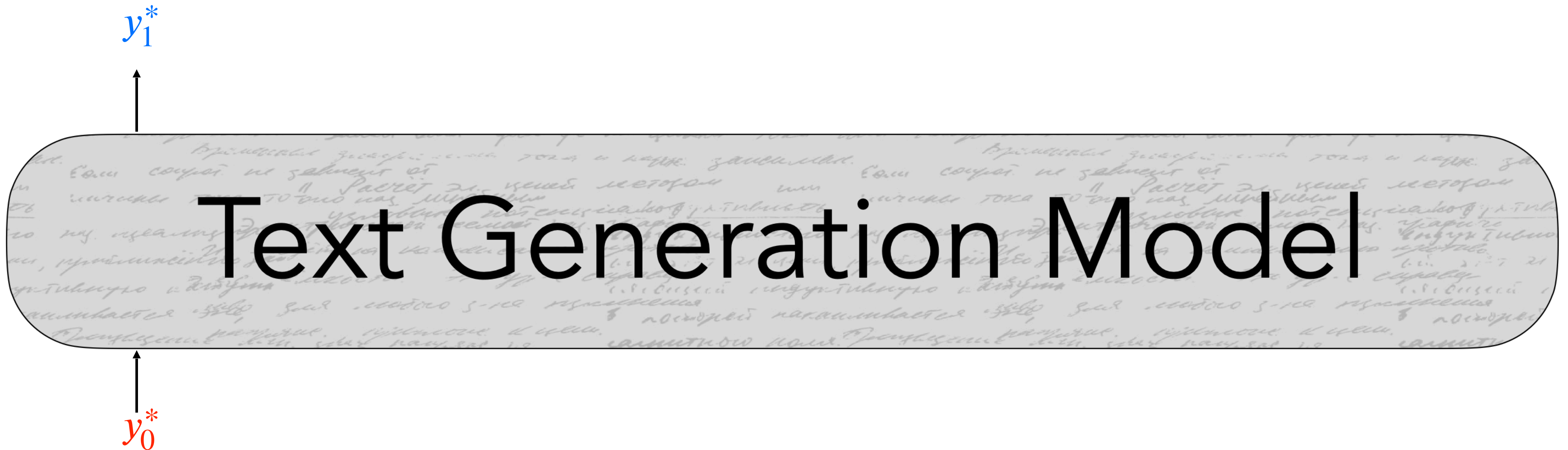
$$\mathcal{L}_t = -\log P(y_t^* | \{y_{<t}^*\})$$

- This is a **multi-class classification task** where each $w \in V$ is a unique class.
- The label at each step is the actual word y_t^* in the training sequence
- This token is often called the "**gold**" or "**ground truth**" token

Maximum Likelihood Training (i.e., *teacher forcing*)

- Trained to generate the next word y_t^* given a set of preceding words $\{y^*\}_{<t}$

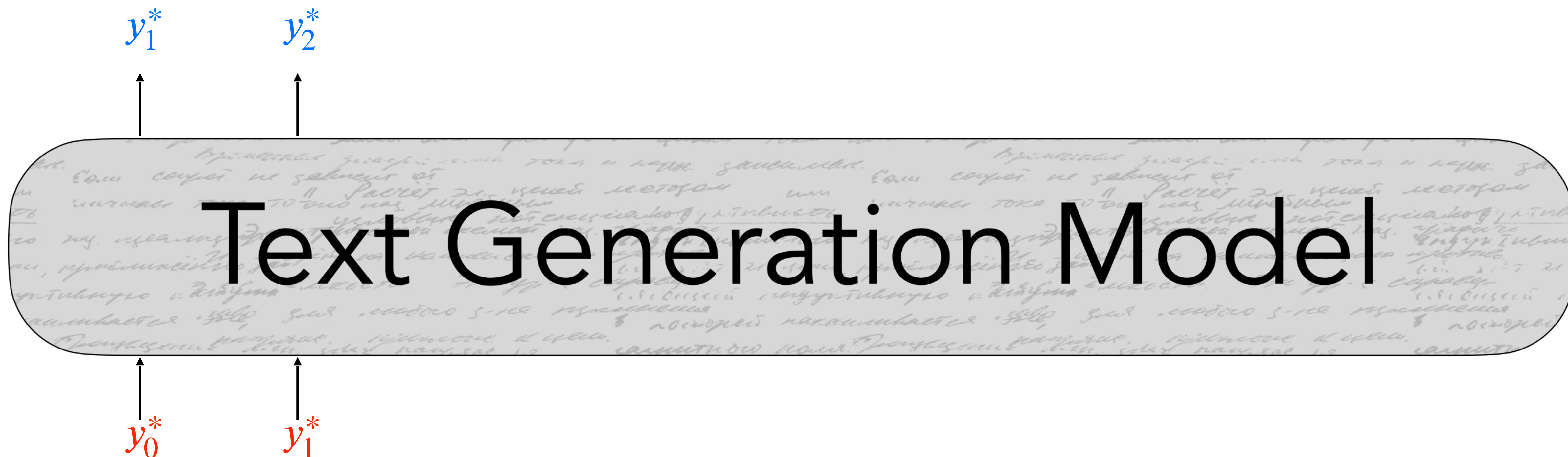
$$\mathcal{L} = -\log P(y_1^* | y_0^*)$$



Maximum Likelihood Training (i.e., *teacher forcing*)

- Trained to generate the next word y_t^* given a set of preceding words $\{y^*\}_{<t}$

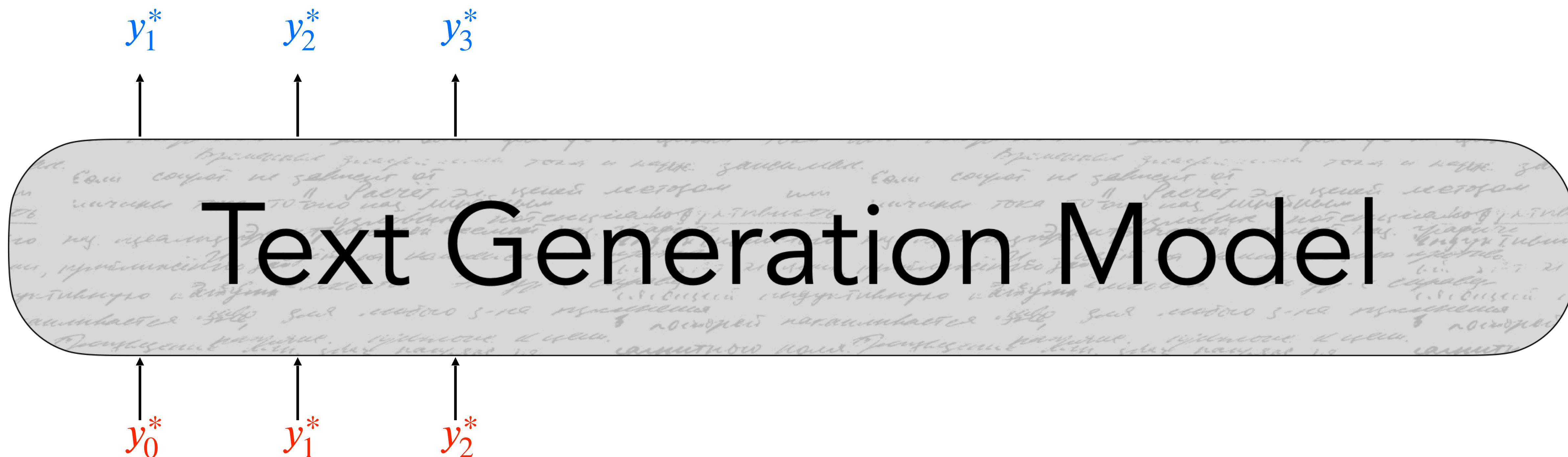
$$\mathcal{L} = - \left(\log P(y_1^* | y_0^*) + \log P(y_2^* | y_0^*, y_1^*) \right)$$



Maximum Likelihood Training (i.e., *teacher forcing*)

- Trained to generate the next word y_t^* given a set of preceding words $\{y^*\}_{<t}$

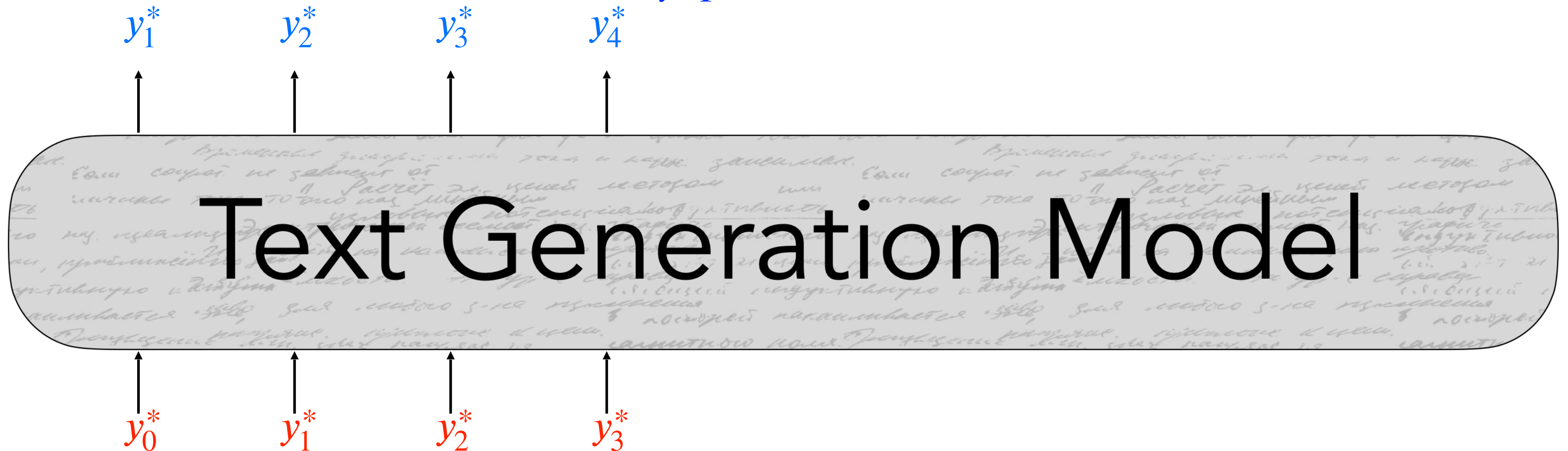
$$\mathcal{L} = - \left(\log P(y_1^* | y_0^*) + \log P(y_2^* | y_0^*, y_1^*) + \log P(y_3^* | y_0^*, y_1^*, y_2^*) \right)$$



Maximum Likelihood Training (i.e., *teacher forcing*)

- Trained to generate the next word y_t^* given a set of preceding words $\{y_{<t}^*\}$

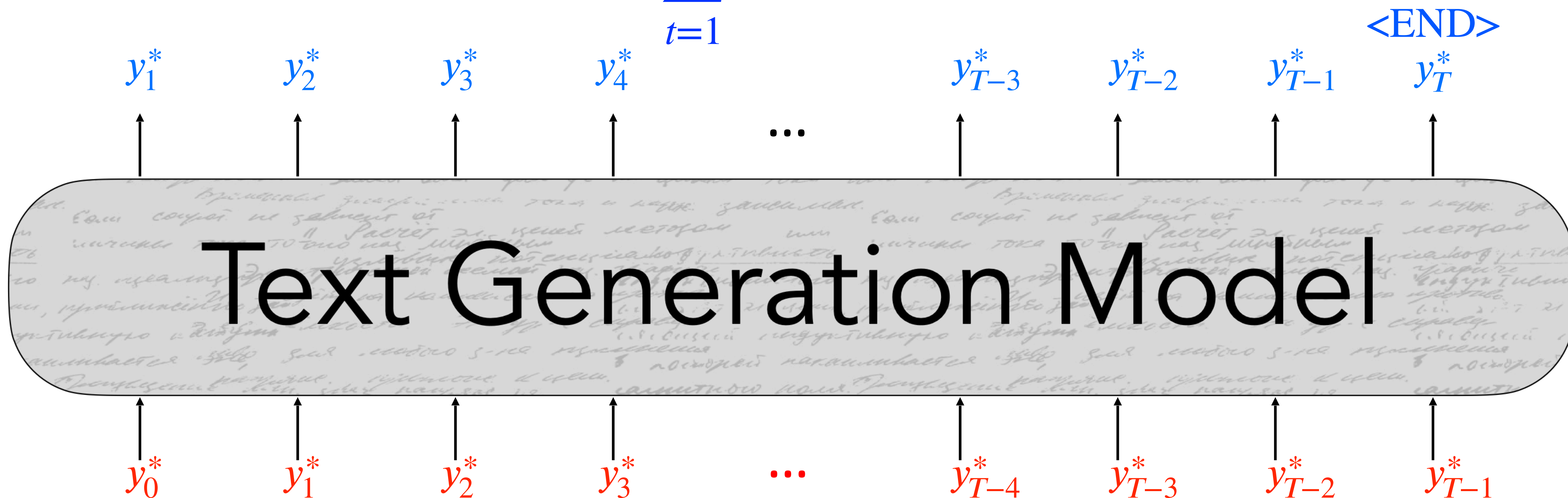
$$\mathcal{L} = - \sum_{t=1}^T \log P(y_t^* | \{y_{<t}^*\})$$



Maximum Likelihood Training (i.e., *teacher forcing*)

- Trained to generate the next word y_t^* given a set of preceding words $\{y_{<t}^*\}$

$$\mathcal{L} = - \sum_{t=1}^T \log P(y_t^* | \{y_{<t}^*\})$$



Basics: What are we trying to do?

- We train the model to minimize the negative loglikelihood of predicting the next token in the sequence:

$$\mathcal{L}_t = -\log P(y_t^* | \{y_{<t}^*\})$$

Sum $\mathcal{L}_t$ for the
entire sequence

- This is a **multi-class classification task** where each $w \in V$ is a unique class.
- The label at each step is the actual word y_t^* in the training sequence
- This token is often called the "**gold**" or "**ground truth**" token
- This algorithm is often called "teacher forcing"

Text Generation: Basics

- In **autoregressive** NLG, we generate one token a time, using the context and previous generated tokens as inputs for generating the next token.
- Our models generate a set of scores for every token in the vocabulary, which we convert to a probability distribution using the softmax function
- A decoding algorithm selects a token from this distribution at each step
- For decoding algorithm to generate meaningful tokens, our distribution must be calibrated, so we train our model using maximum likelihood estimation to predict the next token given sequences from a dataset

Decoding: what is it all about?

- At each time step t , our model computes a vector of scores for each token in our vocabulary, $\mathbf{S} \in \mathbb{R}^V$:

$$\mathbf{S} = f\left(\{y_{<t}\}\right)$$

$f(\cdot)$ is your model

- Then, we compute a probability distribution P over these scores (usually with a softmax function):

$$P\left(y_t = w \mid \{y_{<t}\}\right) = \frac{\exp(\mathbf{S}_w)}{\sum_{w' \in V} \exp(\mathbf{S}_{w'})}$$

- Our decoding algorithm defines a function to select a token from this distribution:

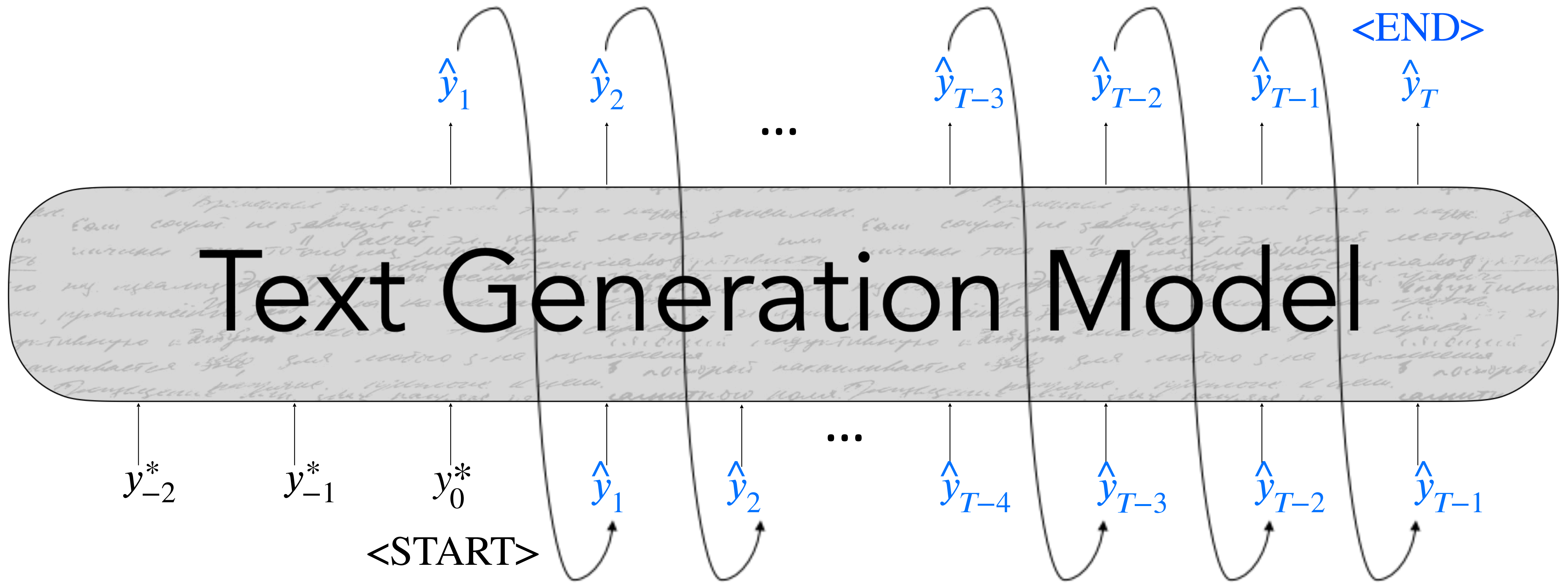
$$\hat{y}_t = g\left(P(y_t \mid \{y_{<t}\})\right)$$

$g(\cdot)$ is your decoding algorithm

Decoding: what is it all about?

- Our decoding algorithm defines a function to select a token from this distribution

$$\hat{y}_t = g(P(y_t | \{y^*\}, \hat{y}_{<t}))$$



Greedy methods: Argmax Decoding

$$\hat{y}_t = \underset{w \in V}{\operatorname{argmax}} P(y_t = w \mid \{y\}_{<t})$$

Select **highest scoring** token

- g = select the token with the highest probability:

He wanted to go to the

Model

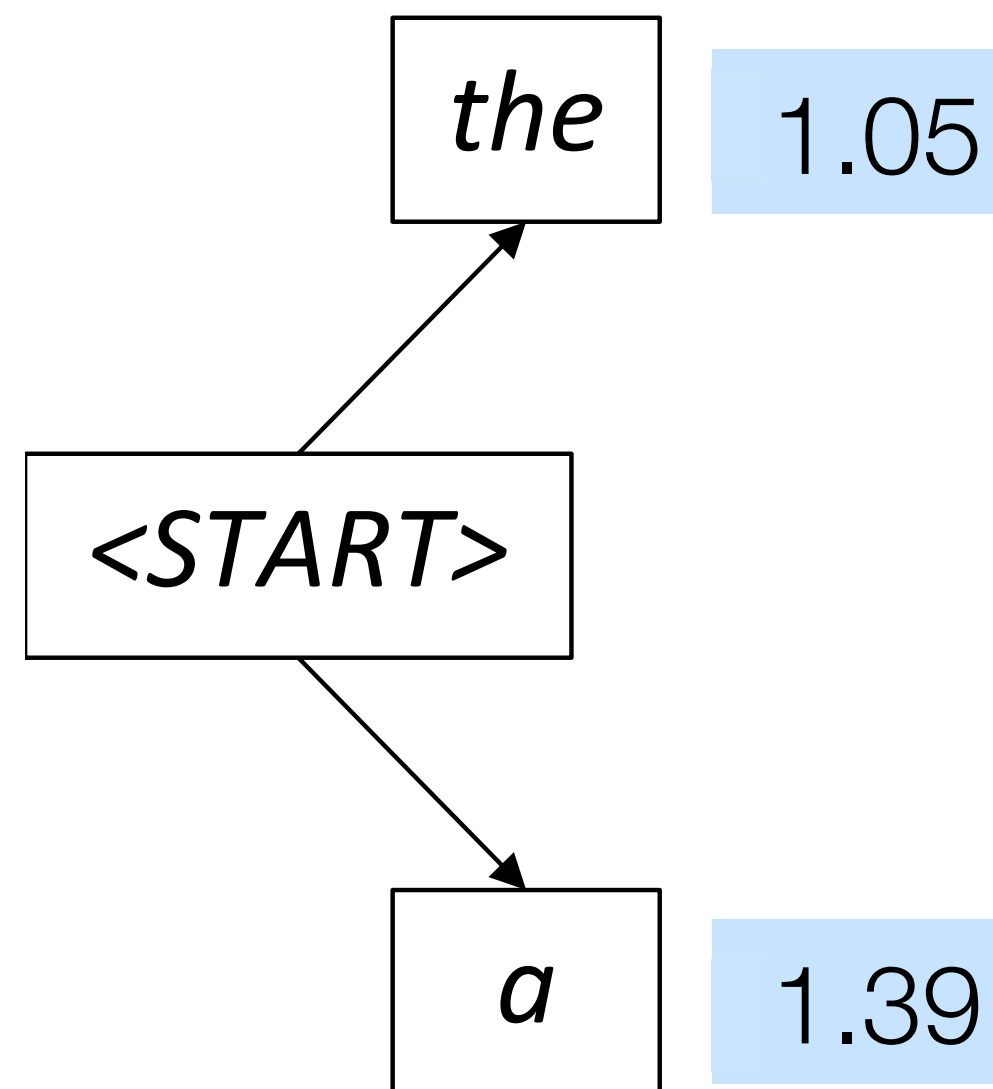


Greedy methods: Beam Search

- In greedy decoding, we cannot revise prior decisions
 - *les pauvres sont démunis (the poor don't have any money)*
 - → *the* _____
 - → *the poor* _____
 - → *the poor* **are** _____
- Beam Search: Explore several different hypotheses instead of just one
 - Keep track of the the b most probable sequences at each decoder step instead of just one
 - b is called the **beam size**

Greedy methods: Beam Search

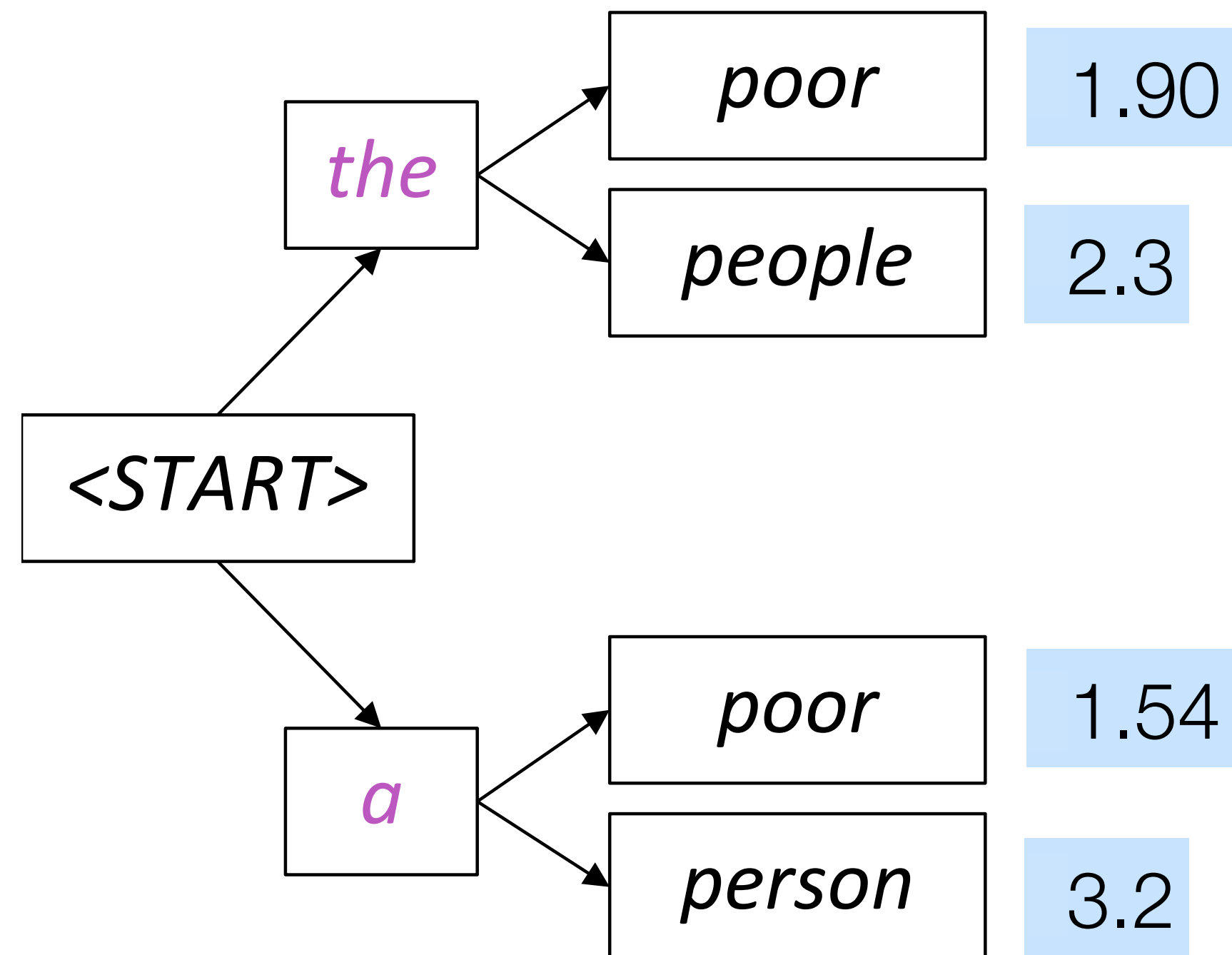
Beam size = 2



Greedy methods: Beam Search

Beam size = 2

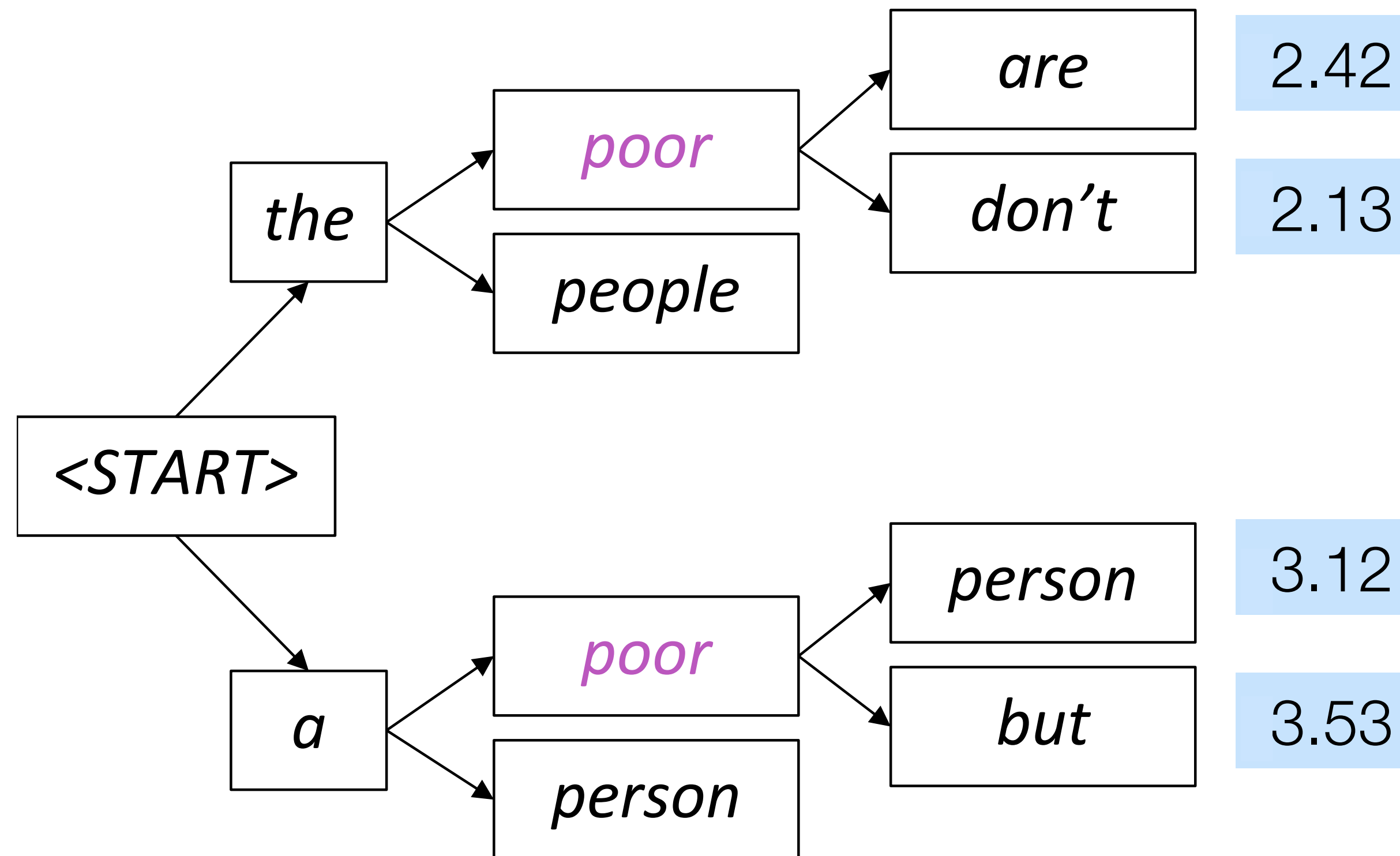
$$-\sum_{t=1}^2 \log P(\hat{y}_t | \hat{y}_0, \dots, \hat{y}_{t-1})$$



Greedy methods: Beam Search

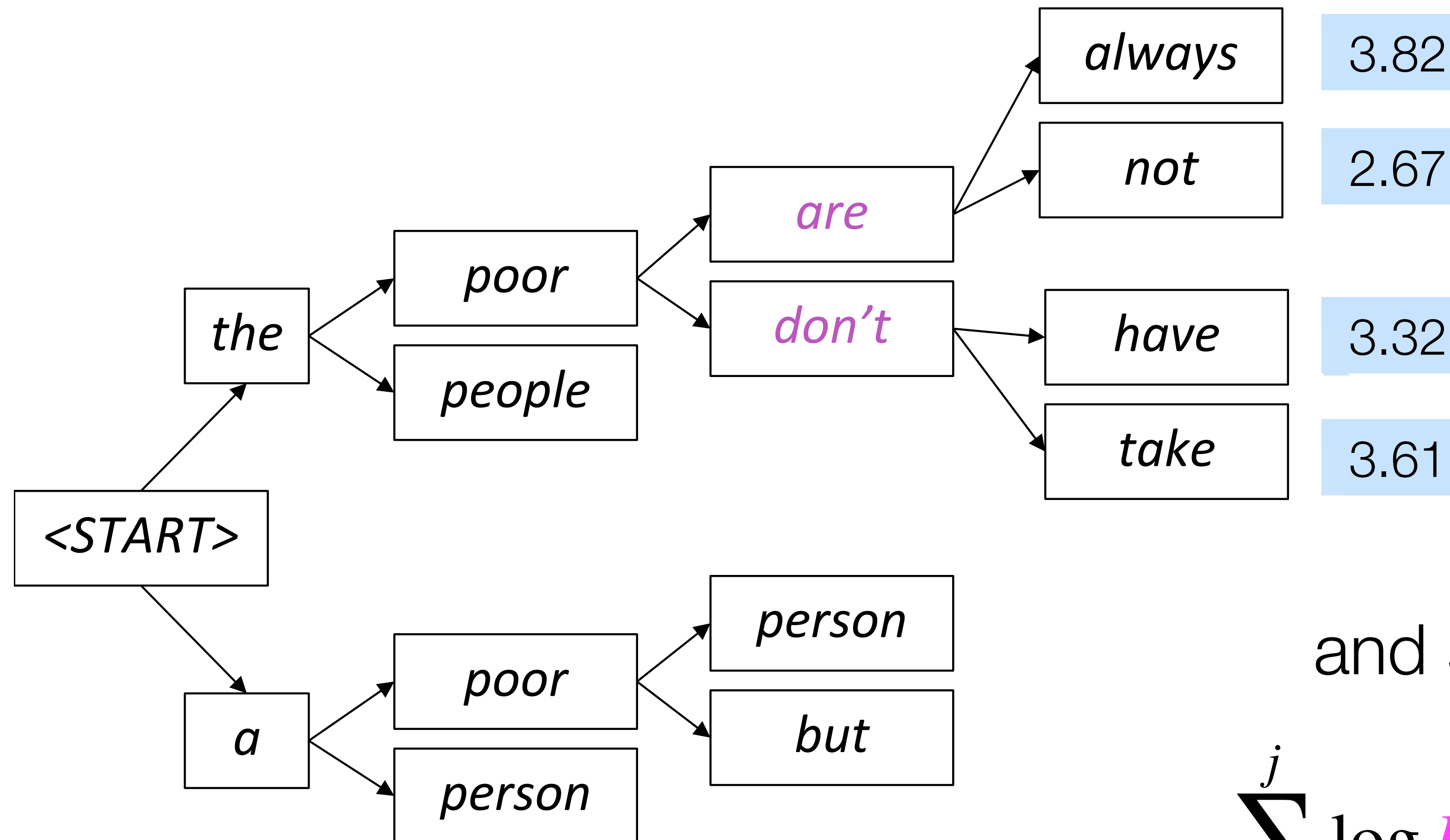
Beam size = 2

$$-\sum_{t=1}^3 \log P(\hat{y}_t | y_0, \hat{y}_1, \dots, \hat{y}_{t-1})$$



Greedy methods: Beam Search

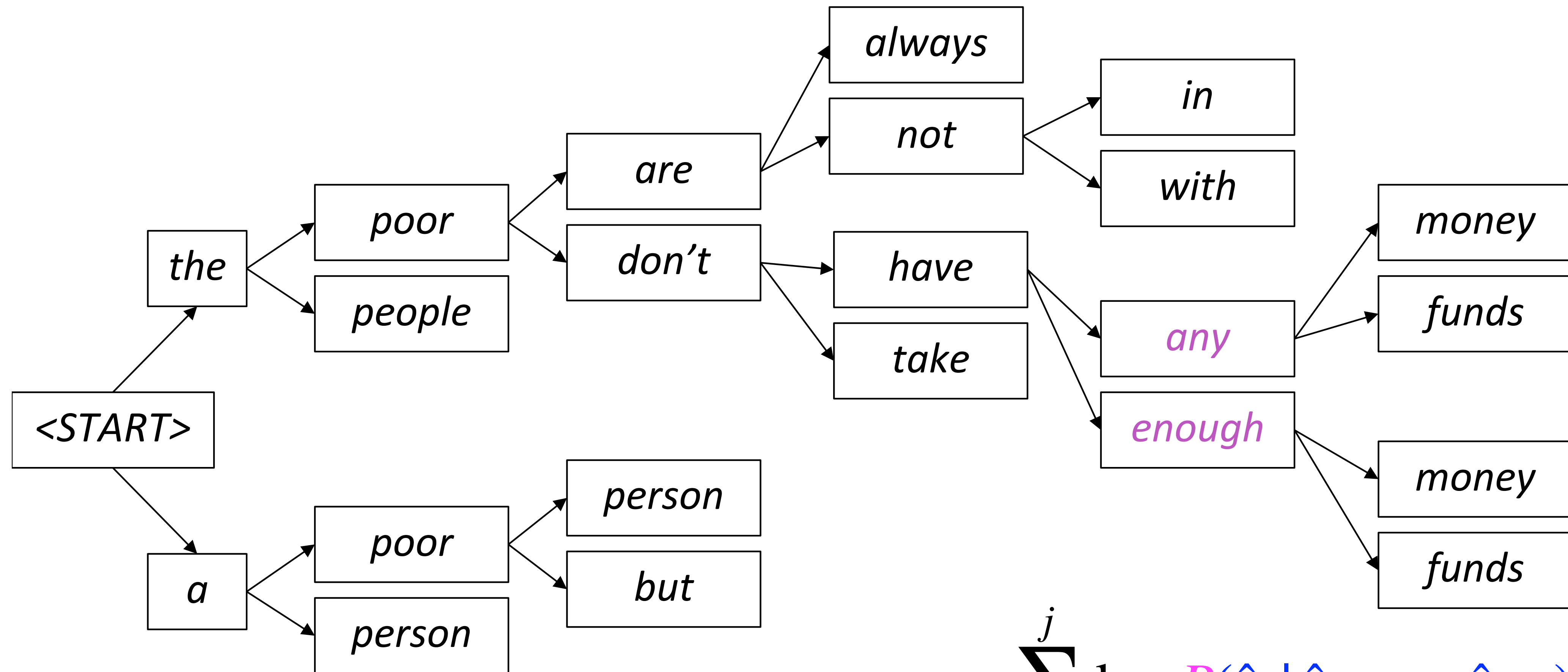
Beam size = 2



$$-\sum_{t=1}^j \log P(\hat{y}_t | \hat{y}_1, \dots, \hat{y}_{t-1})$$

Greedy methods: Beam Search

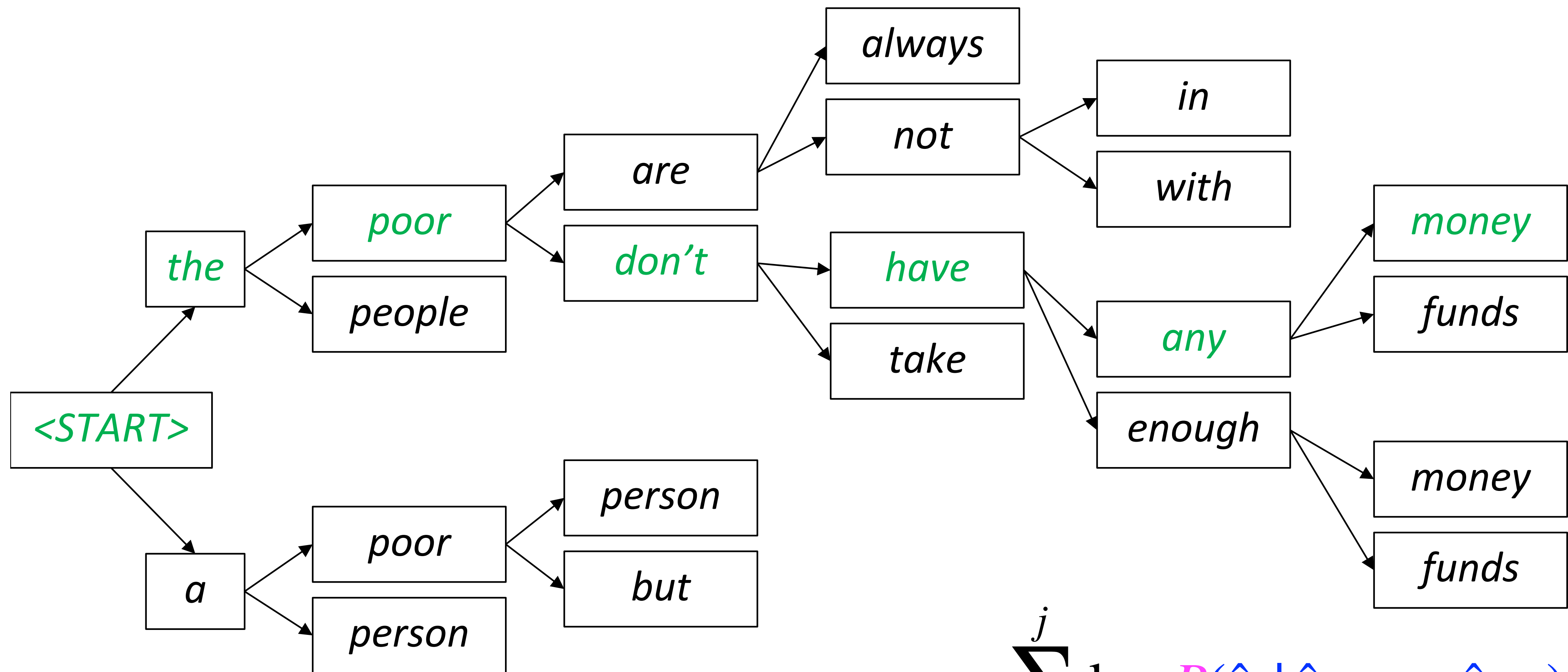
Beam size = 2



$$-\sum_{t=1}^j \log P(\hat{y}_t | \hat{y}_1, \dots, \hat{y}_{t-1})$$

Greedy methods: Beam Search

Beam size = 2

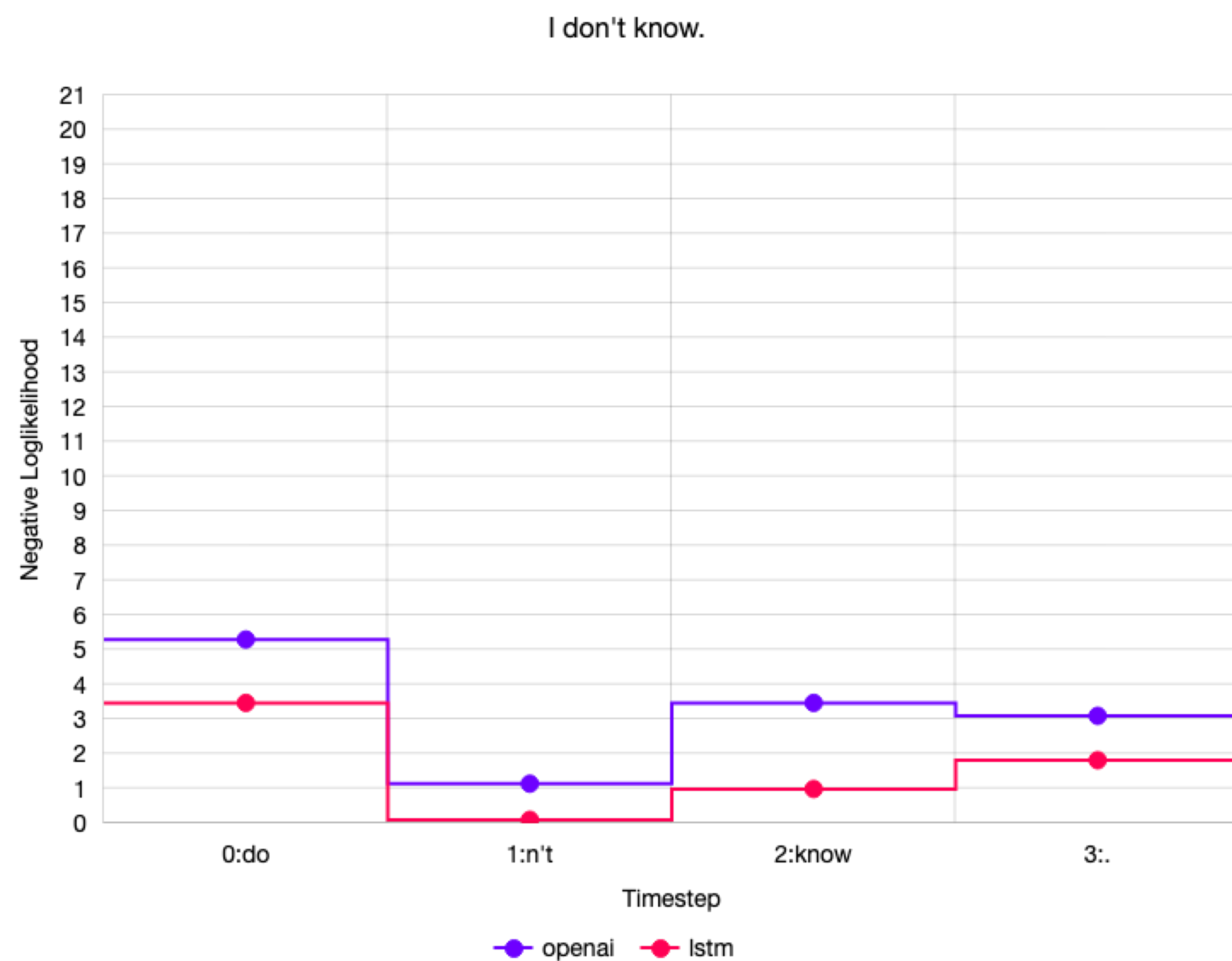


$$-\sum_{t=1}^j \log P(\hat{y}_t | \hat{y}_1, \dots, \hat{y}_{t-1})$$

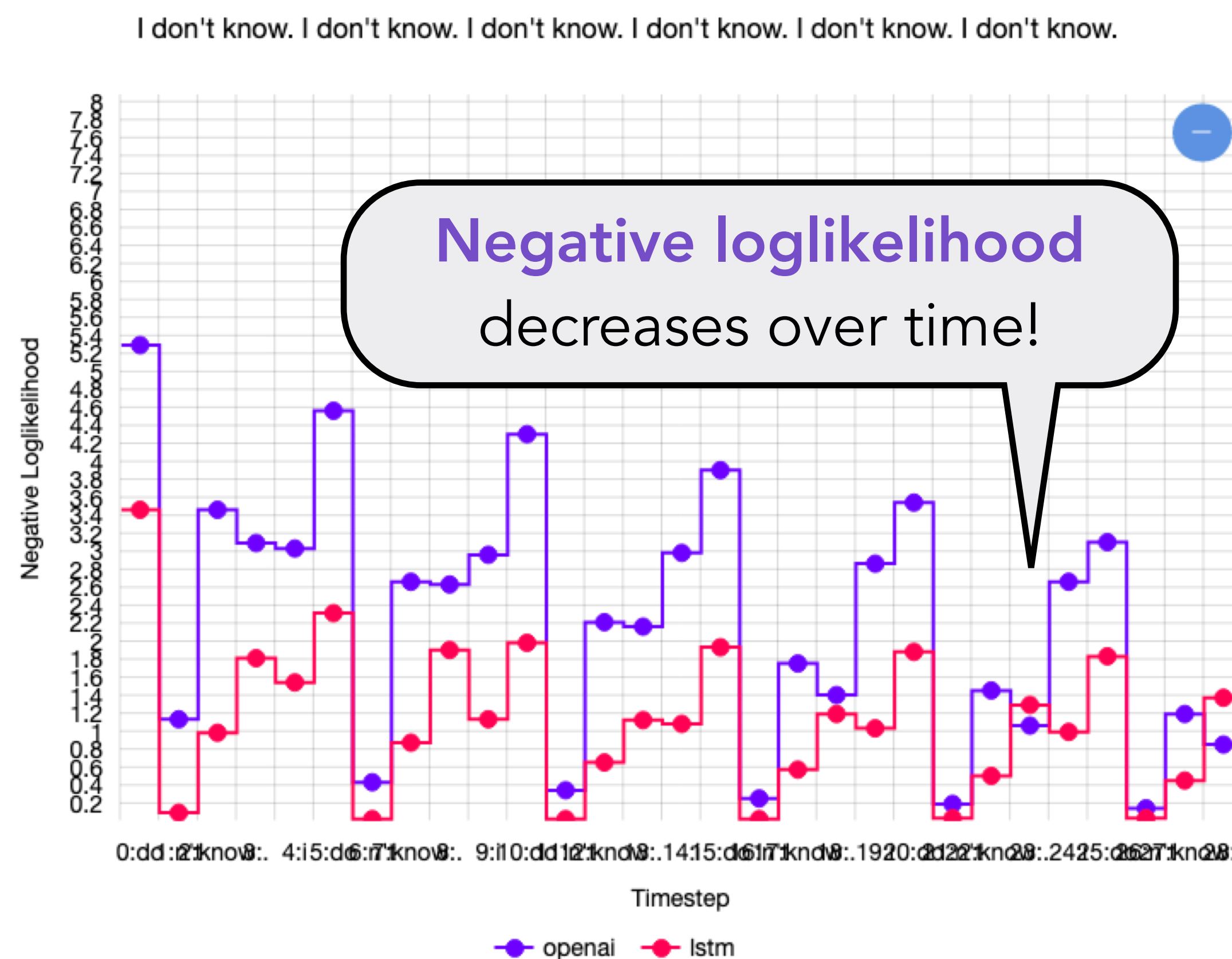
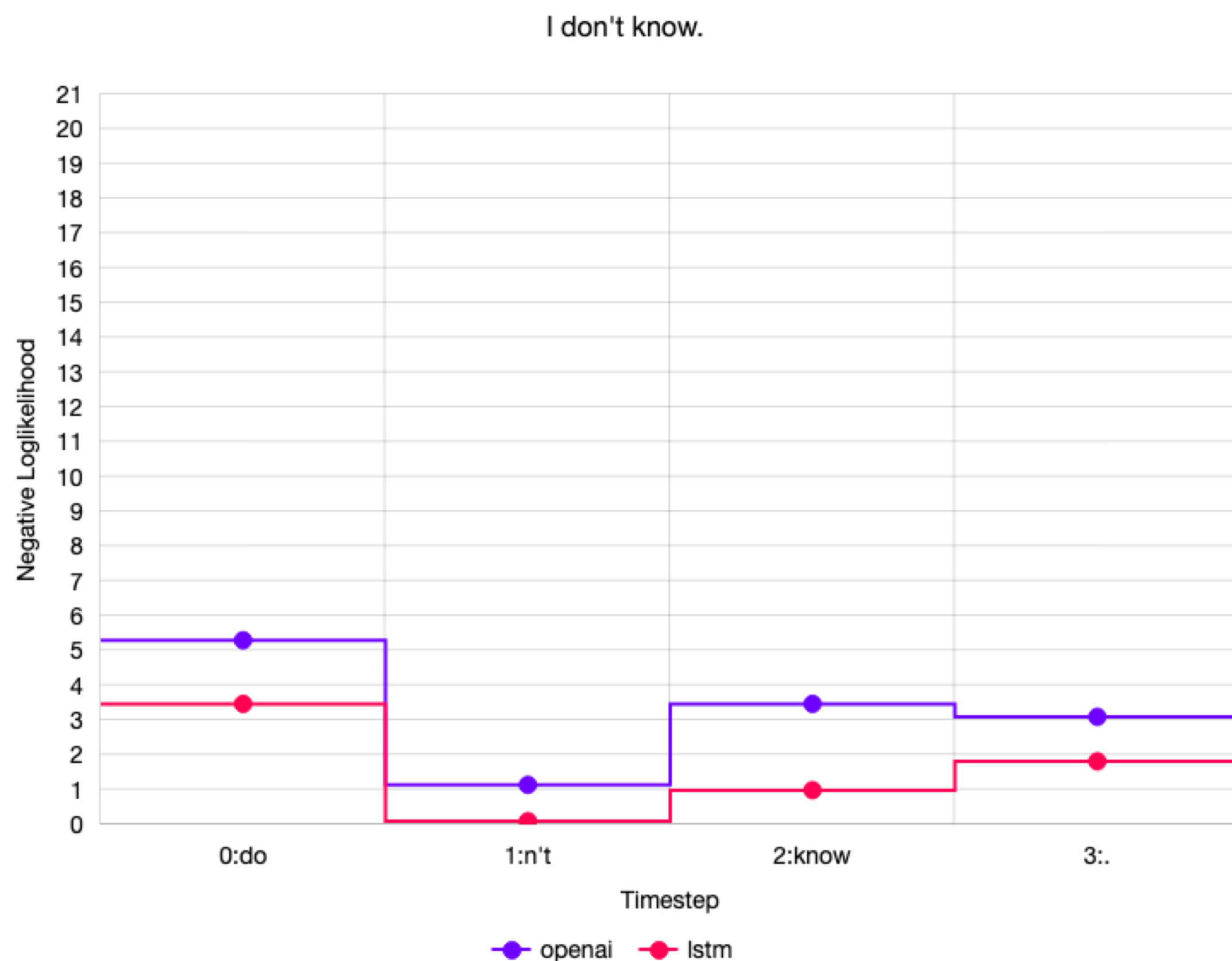
What do you think might be an inherent problem with greedy decoding?

**They maximise the likelihood of the sequence.
What do maximum likelihood sequences look like?**

Step-by-step NLLs



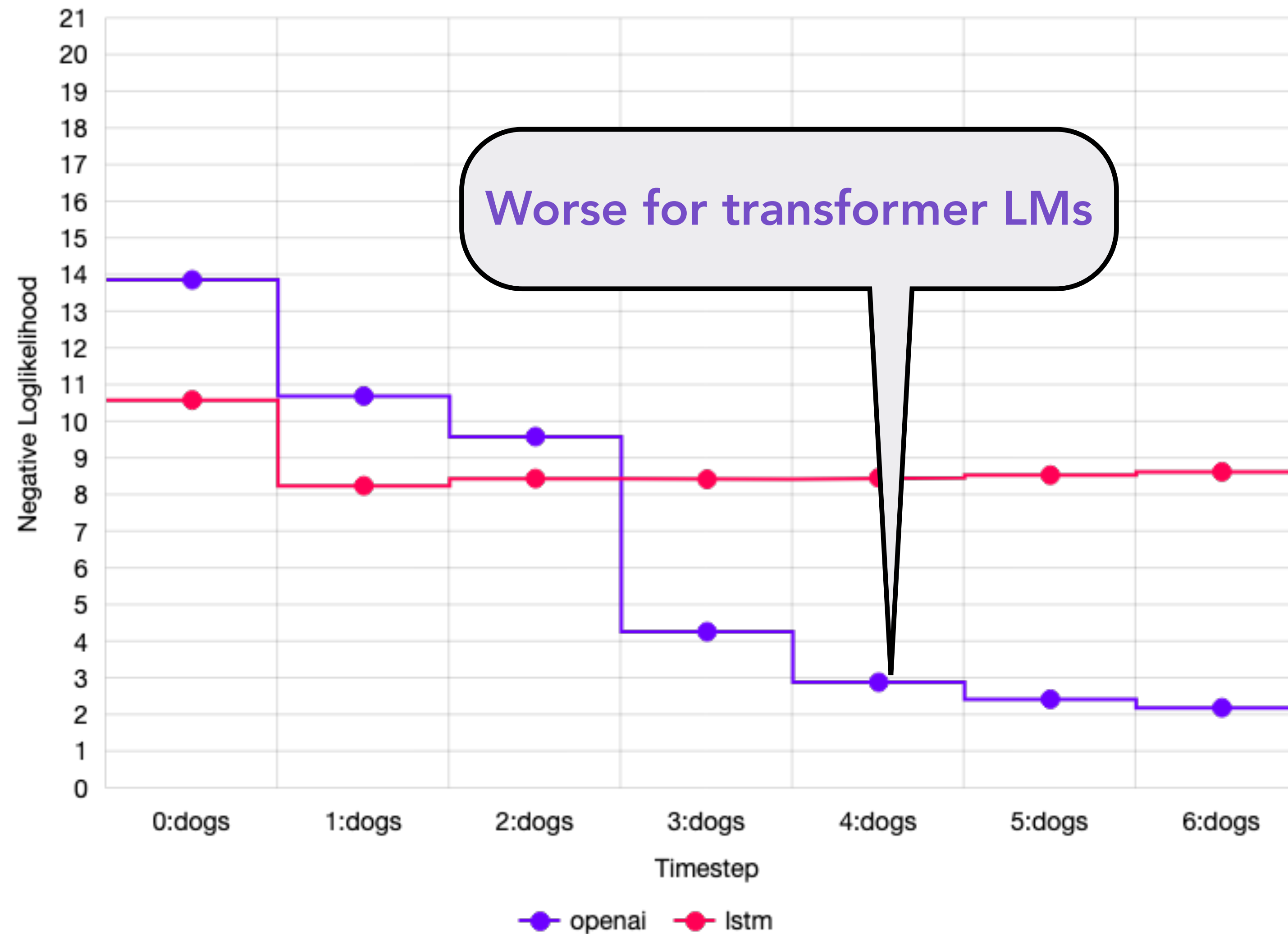
Step-by-step NLLs



What do you notice as the number of time steps increases?

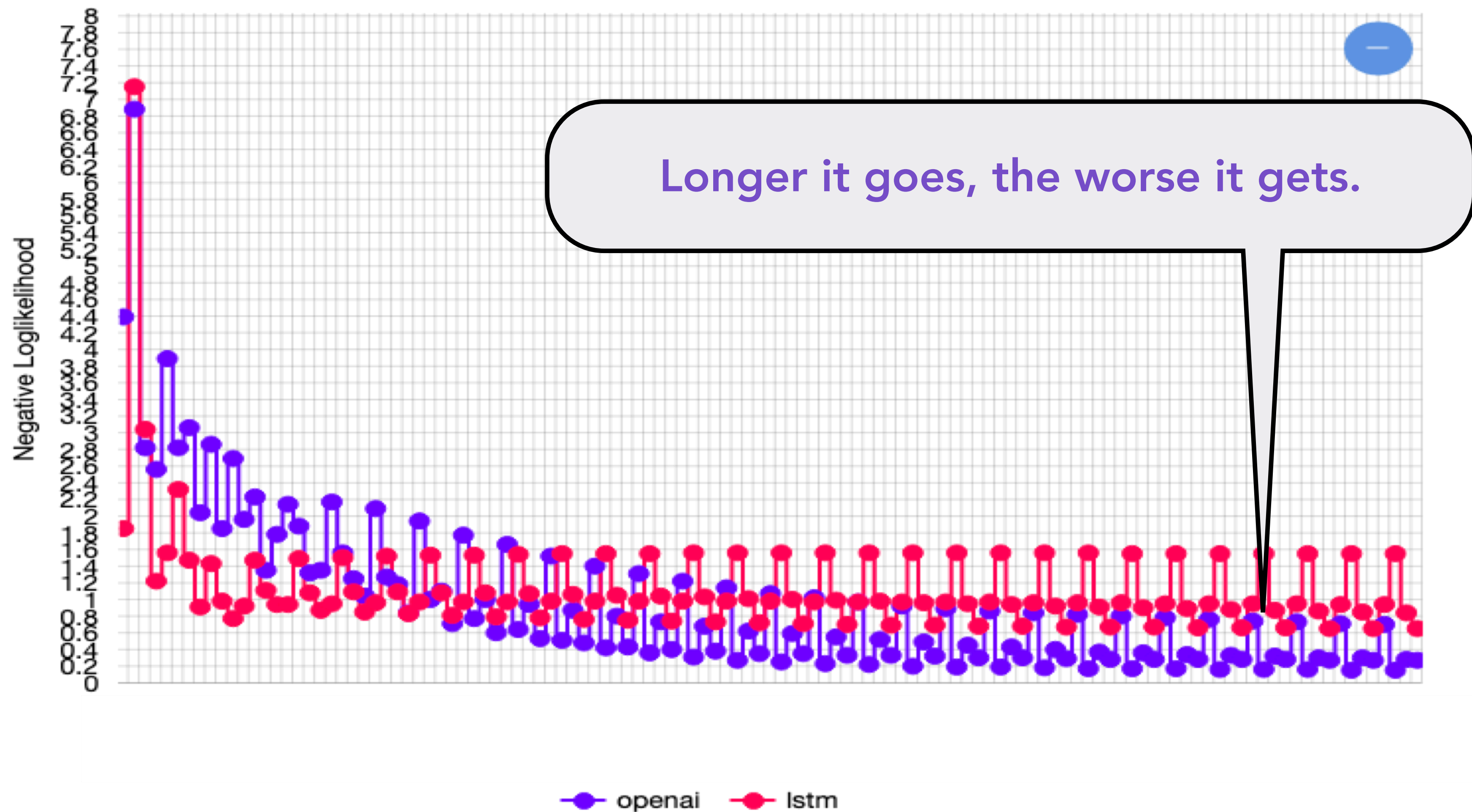
Step-by-step NLLs

dogs dogs dogs dogs dogs dogs dogs dogs



And it keeps going...

I'm tired. I'm tired. I'm tired. I'm tired. I'm tired. I'm tired. I'm tired. I'm tired. I'm tired. I'm tired. I'm tired.



Greedy methods get repetitive

Context: In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.

Continuation: The study, published in the Proceedings of the National Academy of Sciences of the United States of America (PNAS), was conducted by researchers from the **Universidad Nacional Autónoma de México (UNAM)** and **the Universidad Nacional Autónoma de México (UNAM/Universidad Nacional Autónoma de México/ Universidad Nacional Autónoma de México/ Universidad Nacional Autónoma de México/ Universidad Nacional Autónoma de México...** (Holtzman et. al., ICLR 2020)

Greedy methods get repetitive

Context: In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.

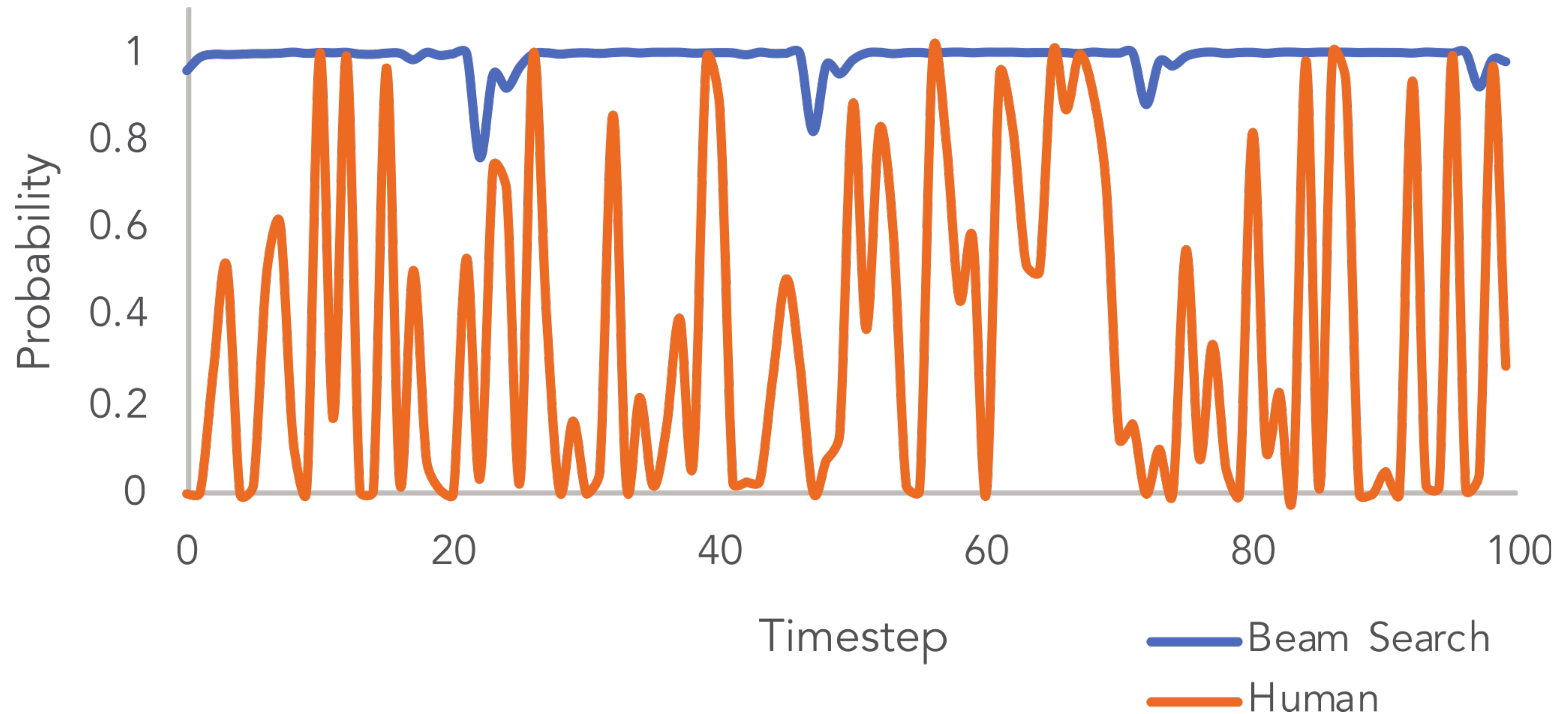
Continuation:

Repetition is a big problem in text generation!

Universidad Nacional Autónoma de México (UNAM) and **the**
Universidad Nacional Autónoma de México
(UNAM/Universidad Nacional Autónoma de México/
Universidad Nacional Autónoma de México/
Universidad Nacional Autónoma de México/
Universidad Nacional Autónoma de México...

(Holtzman et. al., ICLR 2020)

Are greedy methods reasonable?

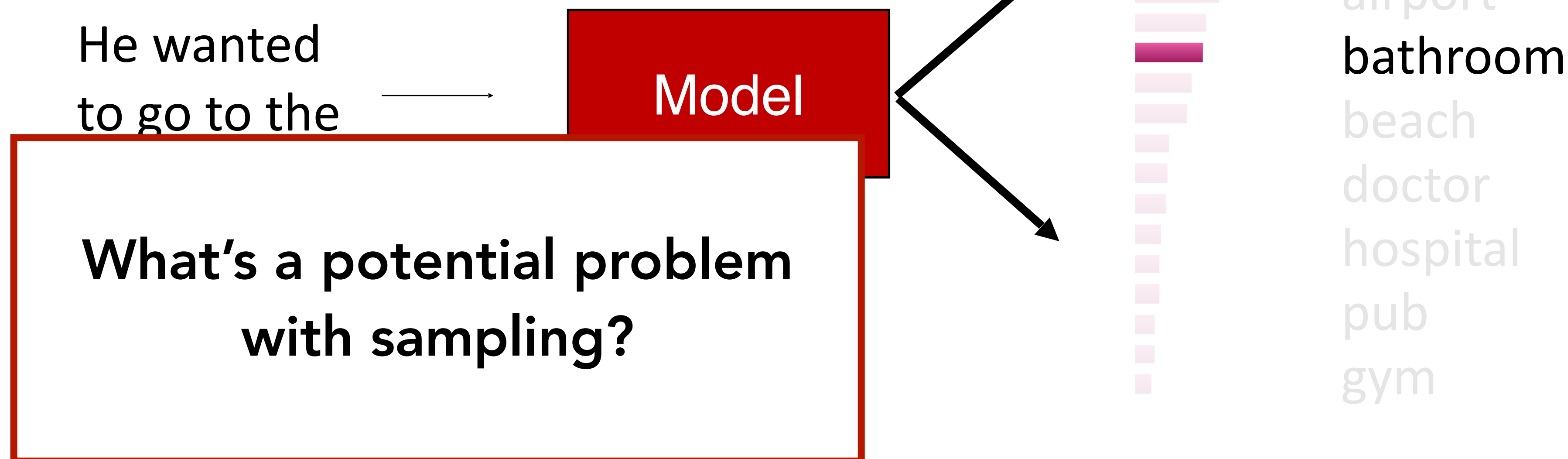


Time to get *random* : Sampling!

- Sample a token from the distribution of tokens

$$\hat{y}_t \sim P(y_t = w \mid \{y\}_{<t})$$

- It's *random* so you can sample any token!



Decoding: Top- k sampling

- Problem: Vanilla sampling makes every token in the vocabulary an option
 - Even if most of the **probability mass** in the distribution is over a limited set of options, the **tail of the distribution could be very long**
 - Many tokens are probably irrelevant in the current context
 - Why are we giving them *individually* a tiny chance to be selected?
 - Why are we giving them *as a group* a high chance to be selected?

Decoding: Top- k sampling

- Problem: Vanilla sampling makes every token in the vocabulary an option
 - Even if most of the **probability mass** in the distribution is over a limited set of options, the **tail of the distribution could be very long**
 - Many tokens are probably irrelevant in the current context
 - Why are we giving them *individually* a tiny chance to be selected?
 - Why are we giving them *as a group* a high chance to be selected?
- Solution: Top- k sampling
 - Only sample from the top k tokens in the probability distribution

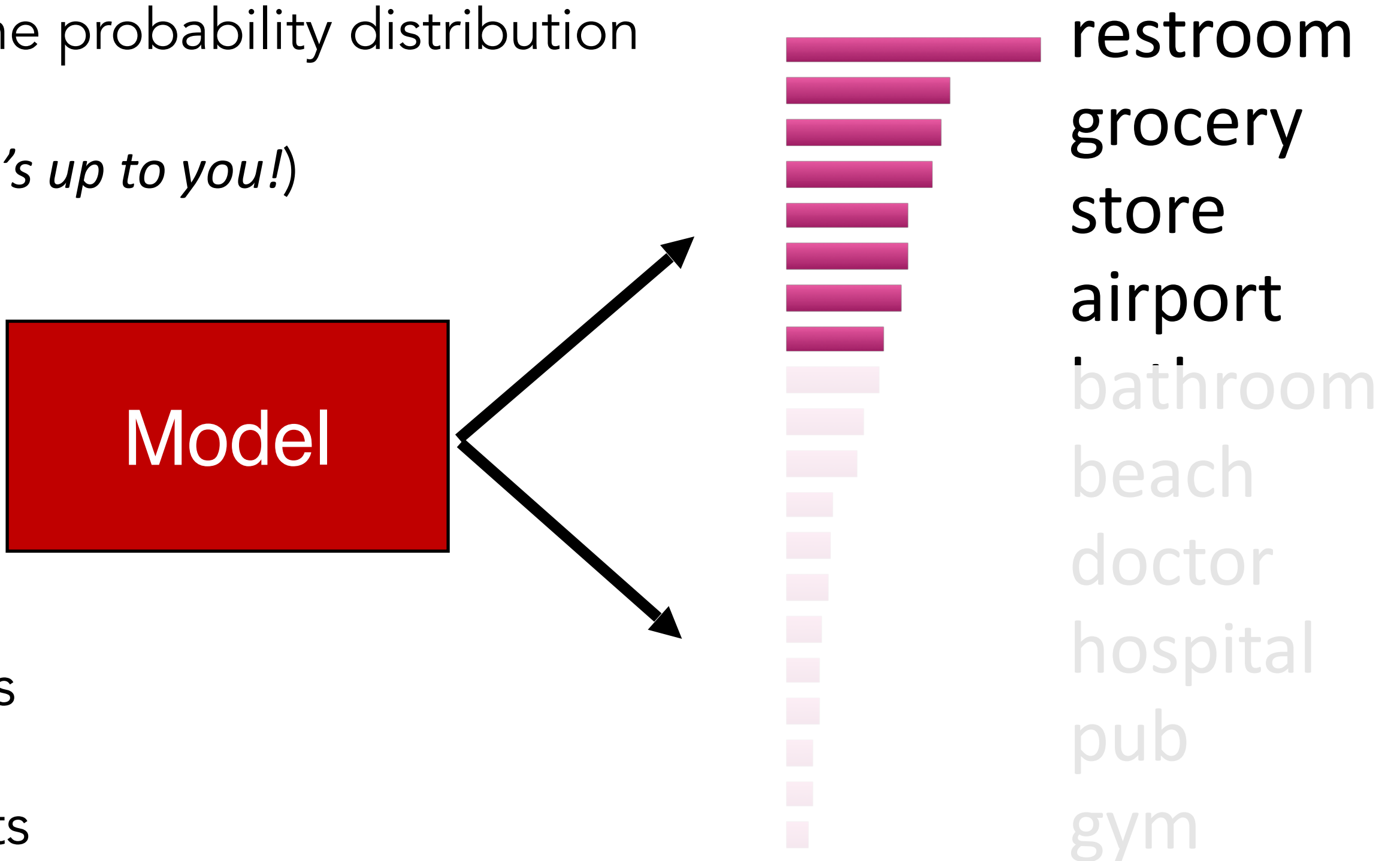
Decoding: Top- k sampling

- Solution: Top- k sampling

- Only sample from the top k tokens in the probability distribution
- Common values are $k = 5, 10, 20$ (*but it's up to you!*)

He wanted
to go to the

Model



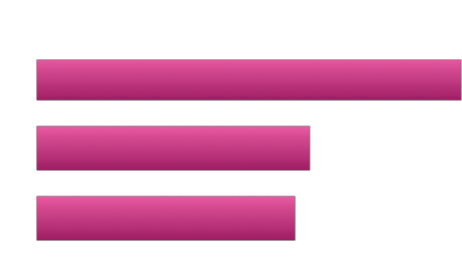
restroom
grocery
store
airport
bathroom
beach
doctor
hospital
pub
gym

- Increase k for more **diverse/risky** outputs
- Decrease k for more **generic/safe** outputs

Decoding: Top- k sampling

- Solution: Top- k sampling

- Only sample from the top k tokens in the probability distribution
- Common values are $k = 5, 10, 20$ (but it's up to you!)



restroom
grocery
rt

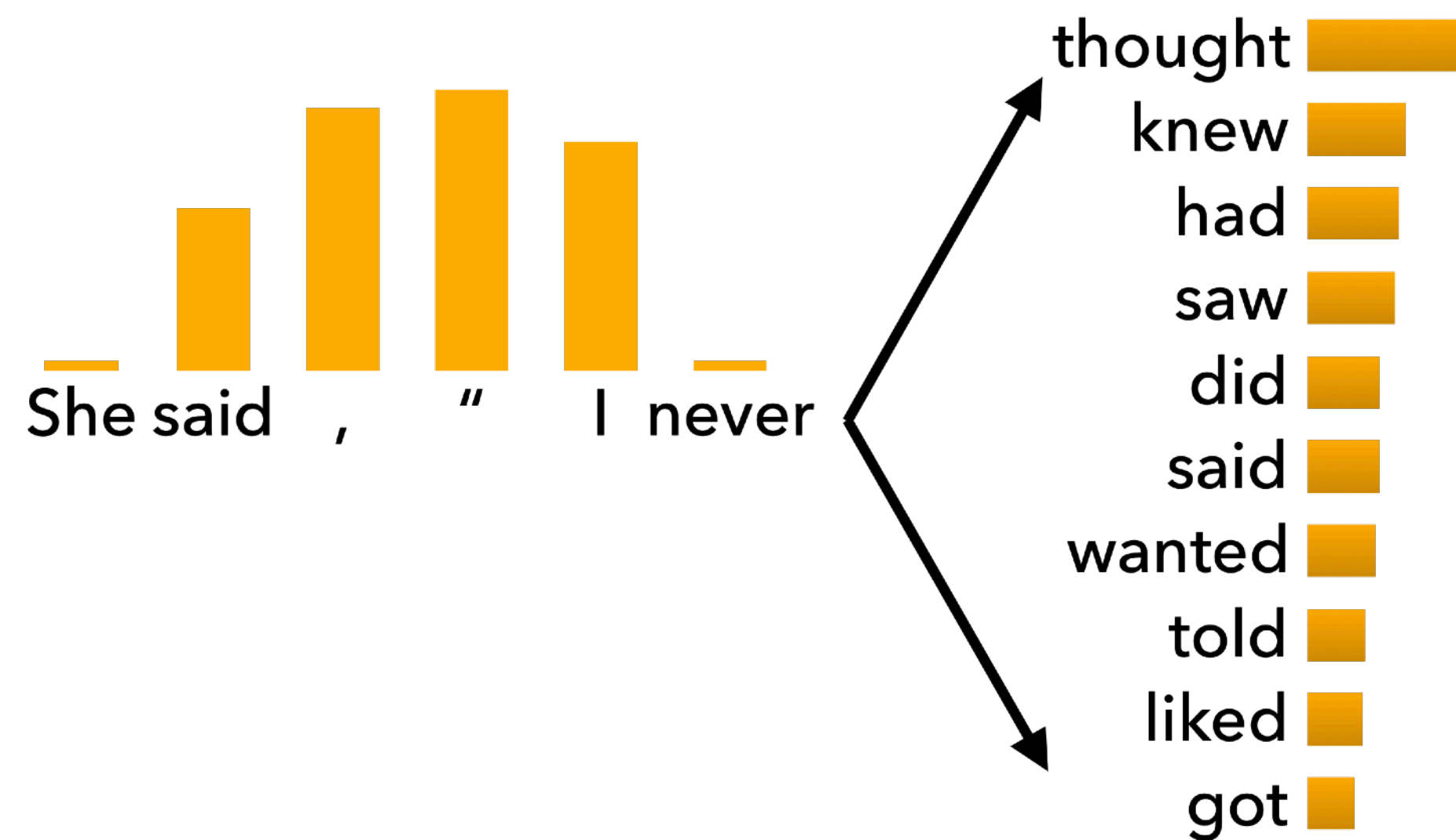
What's a potential problem with top- k sampling?

- Increase k for more **diverse/risky** outputs
- Decrease k for more **generic/safe** outputs

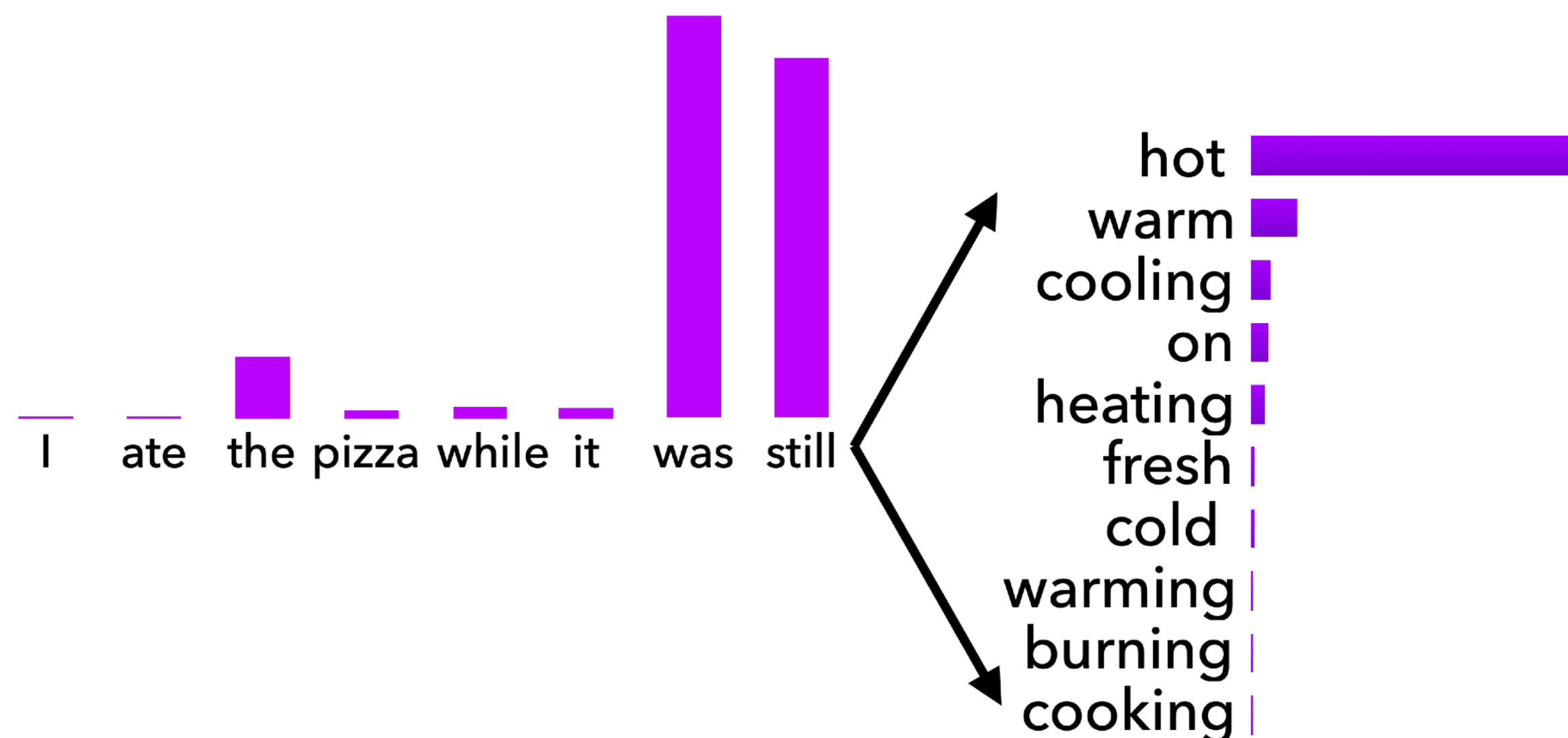


doctor
hospital
pub
gym

Issues with Top- k sampling



Top- k sampling can cut off too ***quickly!***



Top- k sampling can also cut off too ***slowly!***
(same issue as vanilla sampling!)

Decoding: Top- p (nucleus) sampling

- Problem: The probability distributions we sample from are dynamic
 - When the distribution P_t is flatter, a limited k removes many viable options
 - When the distribution P_t is peakier, a high k allows for too many options to have a chance of being selected

Decoding: Top- p (nucleus) sampling

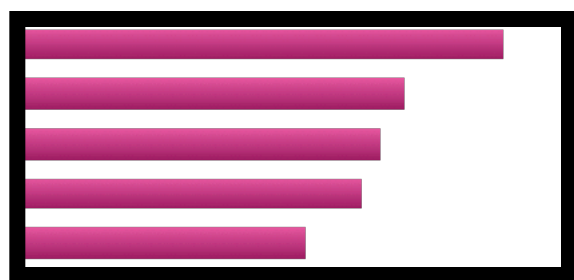
- Problem: The probability distributions we sample from are dynamic
 - When the distribution P_t is flatter, a limited k removes many viable options
 - When the distribution P_t is peakier, a high k allows for too many options to have a chance of being selected
- Solution: Top- p sampling
 - Sample from all tokens in the top p cumulative probability mass (i.e., where mass is concentrated)
 - Varies k depending on the uniformity of P_t

Decoding: Top- p (nucleus) sampling

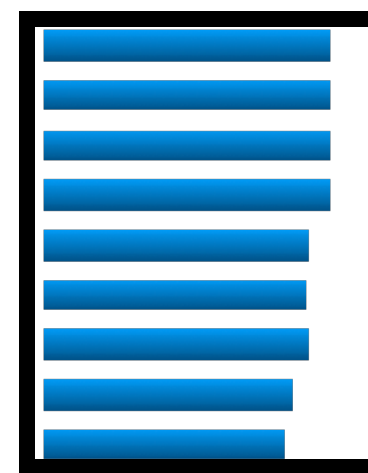
- Solution: Top- p sampling

- Sample from all tokens in the top p cumulative probability mass (i.e., where mass is concentrated)
- Varies k depending on the uniformity of P_t

$$P_t^1(y_t = w \mid \{y\}_{<t})$$



$$P_t^2(y_t = w \mid \{y\}_{<t})$$



$$P_t^3(y_t = w \mid \{y\}_{<t})$$



Scaling randomness: Softmax temperature

- **Recall:** At every time step, the model computes a probability distribution by applying the softmax function to a vector of scores

$$P_t(y_t = w) = \frac{\exp(S_w)}{\sum_{w' \in V} \exp(S_{w'})}$$

- Apply a temperature hyperparameter to the softmax to rebalance :

$$P_t(y_t = w) = \frac{\exp(S_w/\tau)}{\sum_{w' \in V} \exp(S_{w'}/\tau)}$$

What happens if we increase the temperature?

Scaling randomness: Softmax temperature

- **Recall:** At every time step, the model computes a probability distribution by applying the softmax function to a vector of scores

$$P_t(y_t = w) = \frac{\exp(S_w)}{\sum_{w' \in V} \exp(S_{w'})}$$

- Apply a temperature hyperparameter to the softmax to rebalance :

$$P_t(y_t = w) = \frac{\exp(S_w/\tau)}{\sum_{w' \in V} \exp(S_{w'}/\tau)}$$

- Raise the temperature $\tau > 1$:
 - P_t becomes more uniform
 - More diverse output (probability is spread around vocabulary)

What happens if we decrease the temperature?

Scaling randomness: Softmax temperature

- **Recall:** At every time step, the model computes a probability distribution by applying the softmax function to a vector of scores

$$P_t(y_t = w) = \frac{\exp(S_w)}{\sum_{w' \in V} \exp(S_{w'})}$$

- Apply a temperature hyperparameter to the softmax to rebalance :

$$P_t(y_t = w) = \frac{\exp(S_w/\tau)}{\sum_{w' \in V} \exp(S_{w'}/\tau)}$$

- Raise the temperature $\tau > 1$:
 - P_t becomes more uniform
 - More diverse output (probability is spread around vocabulary)
- Lower the temperature $\tau < 1$:
 - P_t becomes more spiky
 - Less diverse output (probability is concentrated on top words)

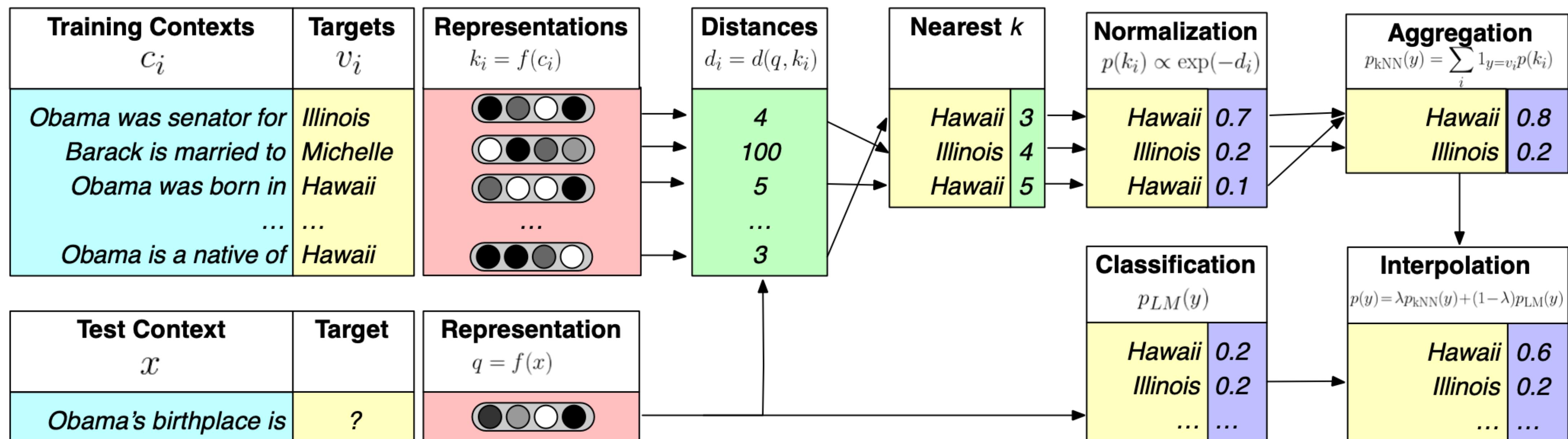
What happens if temperature goes to 0?

$$P_t(y_t = w) = \frac{\exp(S_w/\tau)}{\sum_{w' \in V} \exp(S_{w'}/\tau)}$$

All mass moves to highest probability token —> Argmax Decoding!

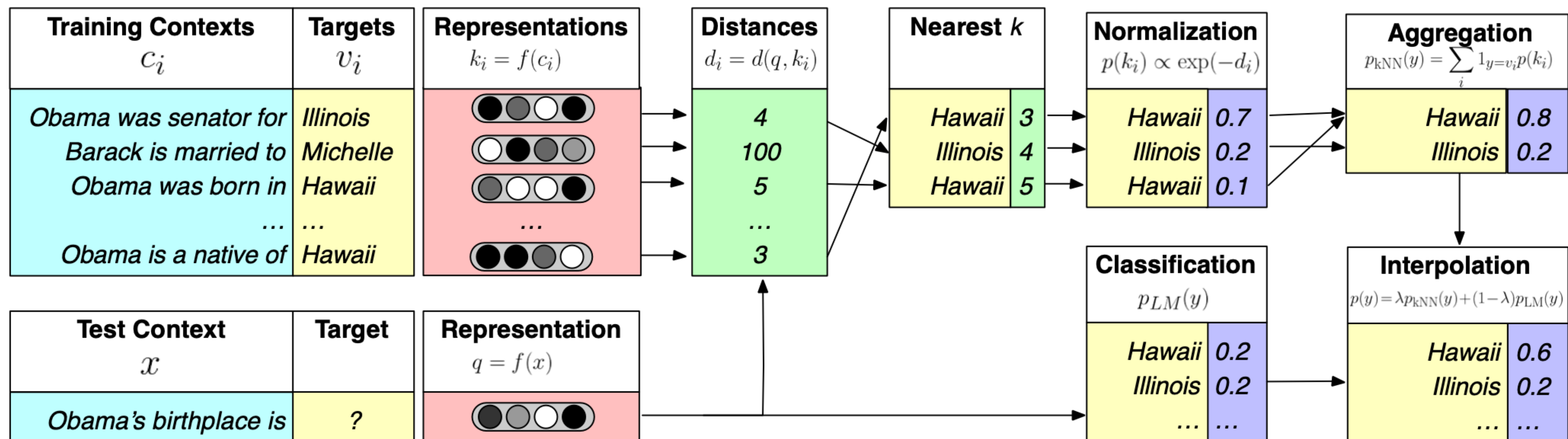
Improving decoding: re-balancing distributions

- Problem: What if I don't trust how well my model's distributions are calibrated?
 - Don't rely on **ONLY** your model's distribution over tokens
- Solution #1: Re-balance P_t using retrieval from n-gram phrase statistics!



Improving decoding: re-balancing distributions

- Solution #1: Re-balance P_t using retrieval from n-gram phrase statistics!
 - Cache a database of phrases from your training corpus (or some other corpus)
 - At decoding time, search for most similar phrases in the database
 - Re-balance P_t using induced distribution P_{phrase} over words that follow these phrases



Improving Decoding: Re-ranking

- **Problem:** What if I decode a bad sequence from my model?
- **Solution:** Decode a bunch of sequences
 - 10 candidates is a common number, but it's up to you
- Define a score to approximate quality of sequences and **re-rank by this score**
 - Simplest is to use **perplexity!**
 - Careful! Remember that **repetitive sequences** can get low perplexity.
 - Re-rankers can score a **variety of properties**:
 - style (Holtzman et al., 2018), discourse (Gabriel et al., 2021), entailment/factuality (Goyal et al., 2020), logical consistency (Lu et al., 2020), and many more...
 - **Beware of poorly-calibrated re-rankers**
 - Can use multiple re-rankers in parallel

Decoding: Takeaways

- Decoding is still a challenging problem in natural language generation
- Human language is not well-approximated by **probability maximization**
- Different decoding algorithms can inject biases that encourage different properties of coherent and aligned natural language generation
- Some of the most **impactful advances** in NLG of the last few years have come from **simple**, but **effective**, modifications to decoding algorithms
- **A lot more work to be done!**

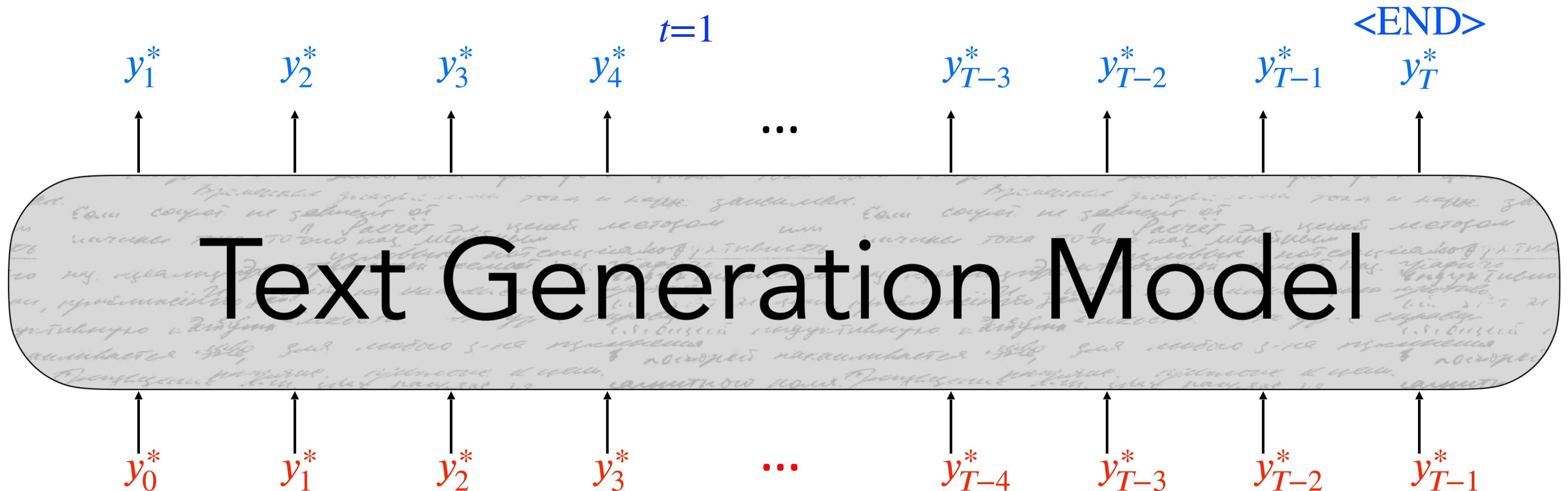
Why do we need these fancy decoders?

**Shouldn't our models just be
better calibrated?**

Maximum Likelihood Training (i.e., *teacher forcing*)

- Trained to generate the next word y_t^* given a set of preceding words $\{y_{<t}^*\}$

$$\mathcal{L} = - \sum_{t=1}^T \log P(y_t^* | \{y_{<t}^*\})$$



Issue #1: MLE *discourages* diversity

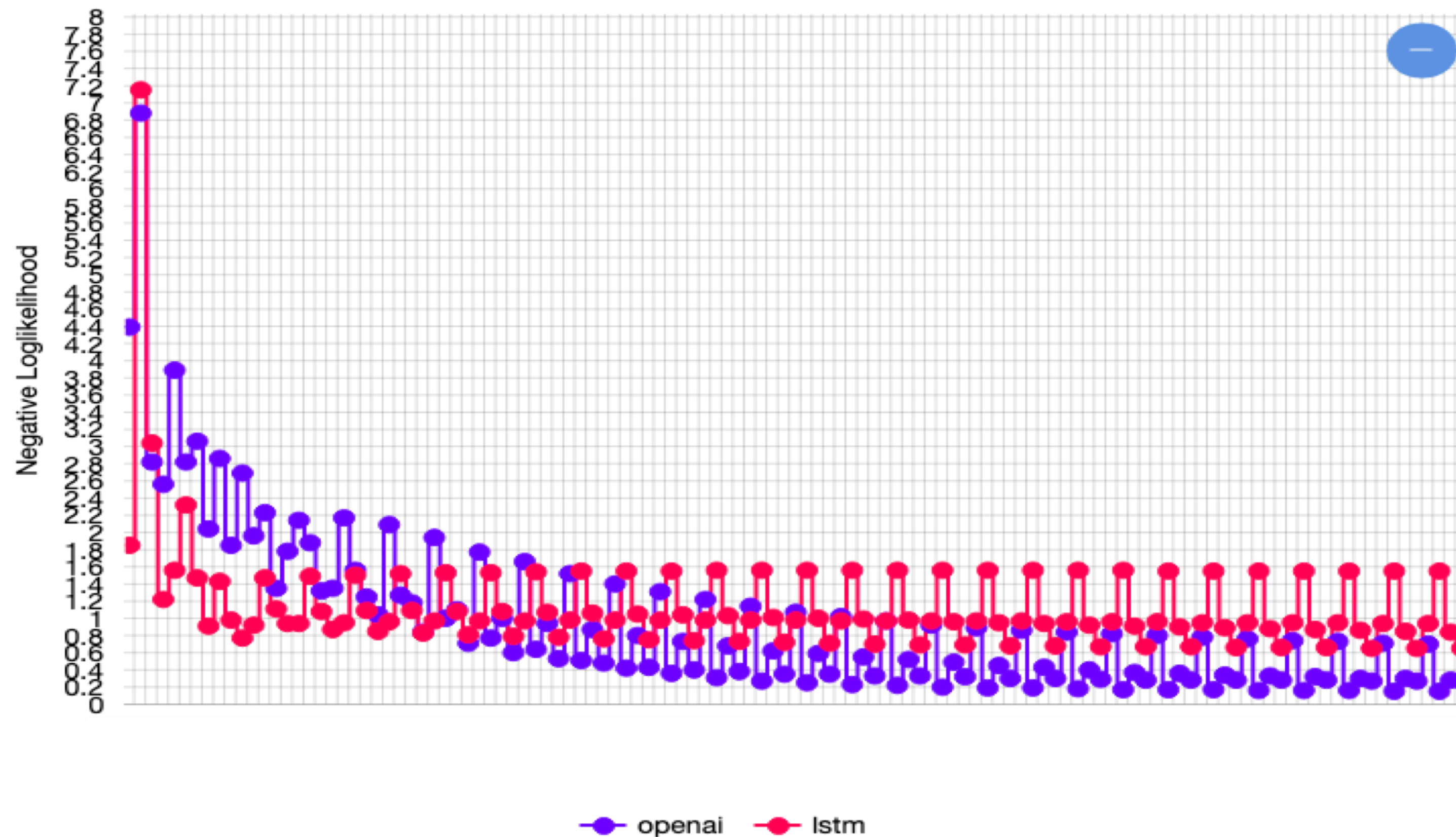
Context: In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.

Continuation: The study, published in the Proceedings of the National Academy of Sciences of the United States of America (PNAS), was conducted by researchers from the **Universidad Nacional Autónoma de México (UNAM)** and **the Universidad Nacional Autónoma de México (UNAM/Universidad Nacional Autónoma de México/ Universidad Nacional Autónoma de México/ Universidad Nacional Autónoma de México/ Universidad Nacional Autónoma de México...**

Issue #1: MLE *discourages* diversity

- Maximum Likelihood Estimation *discourages* diverse text generation

I'm tired. I'm tired. I'm tired. I'm tired. I'm tired. I'm tired. I'm tired. I'm tired. I'm tired. I'm tired. I'm tired.

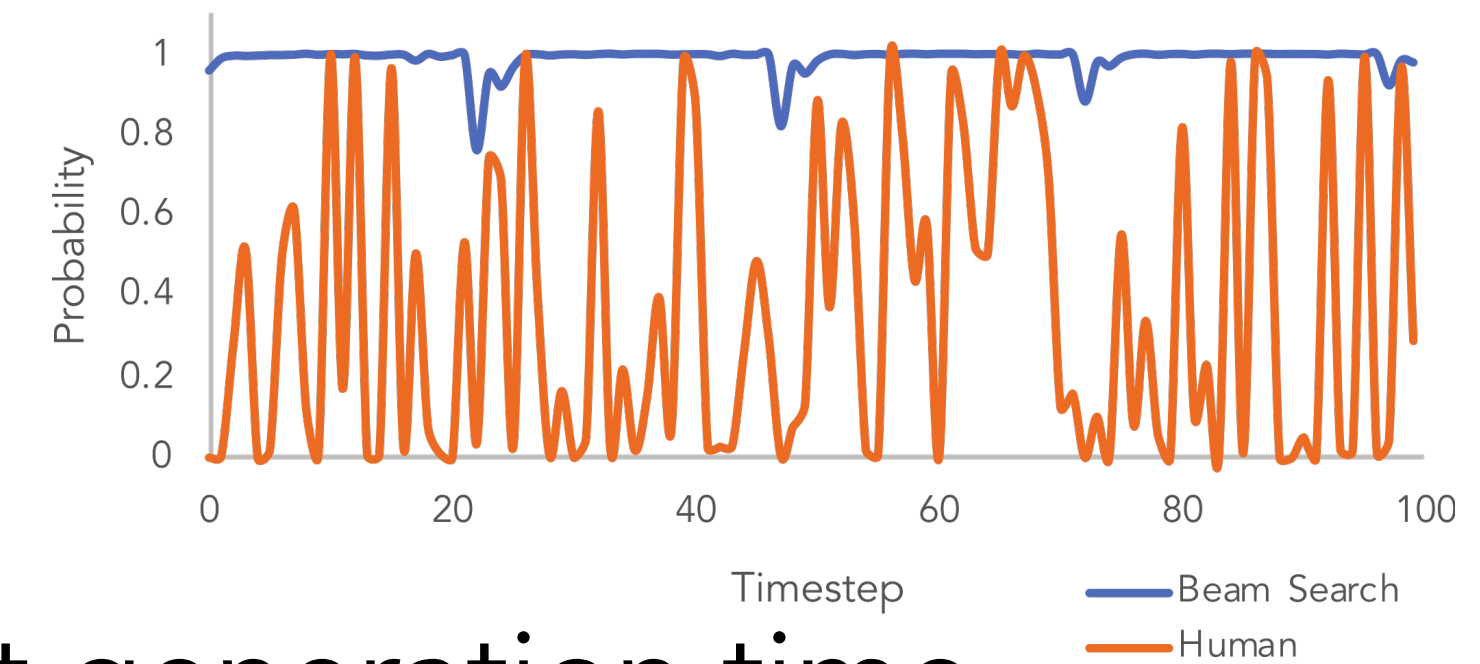


How can we increase the diversity of the sequences that are seen during training?

More data! Pretraining!

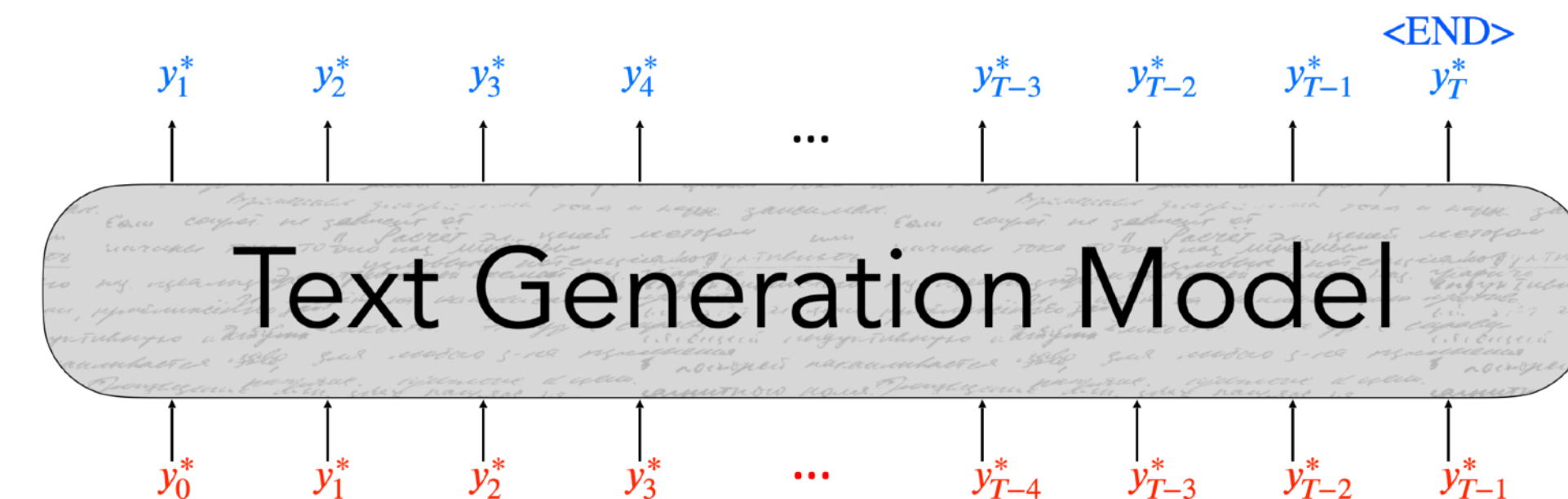
Learn from the sequences the model generates itself!

Issue #2: Exposure Bias



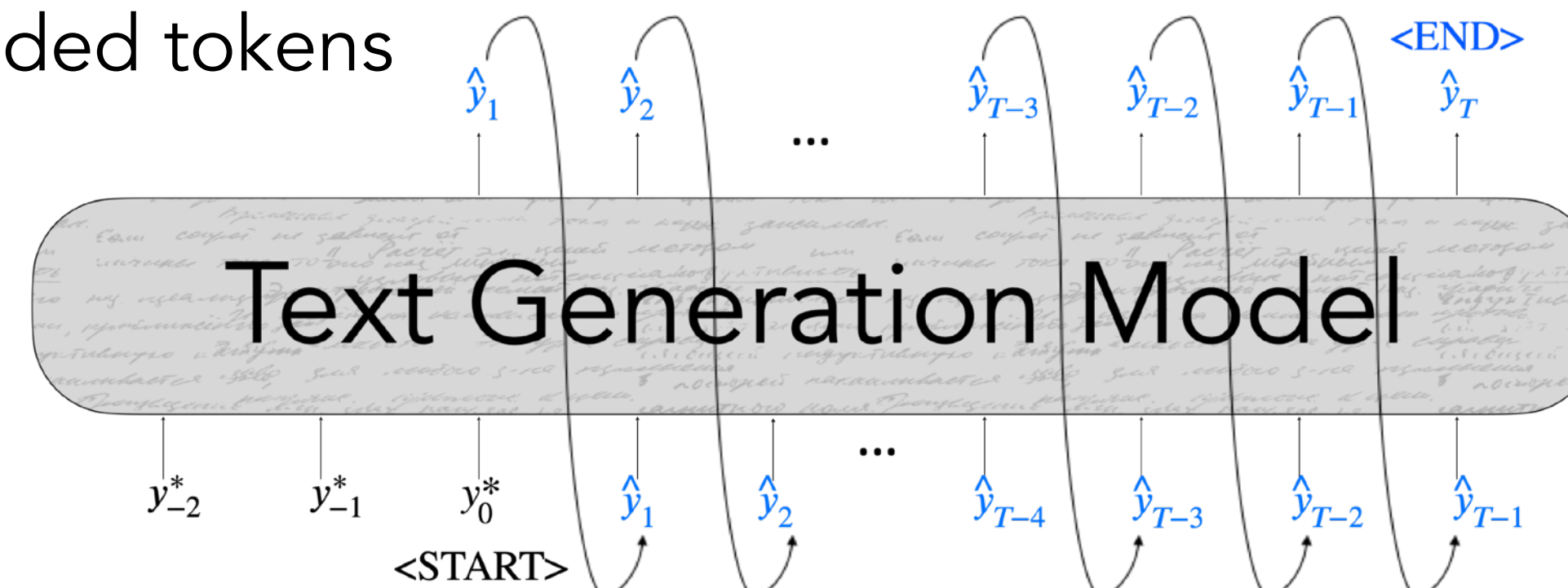
- Training with teacher forcing leads to *exposure bias* at generation time
 - During training, our model's inputs are gold context tokens from real, human-generated texts

$$\mathcal{L}_{MLE} = -\log P(y_t^* | \{y_{<t}^*\})$$



- At generation time, our model's inputs are previously-decoded tokens

$$\mathcal{L}_{dec} = -\log P(\hat{y}_t | \{\hat{y}_{<t}\})$$



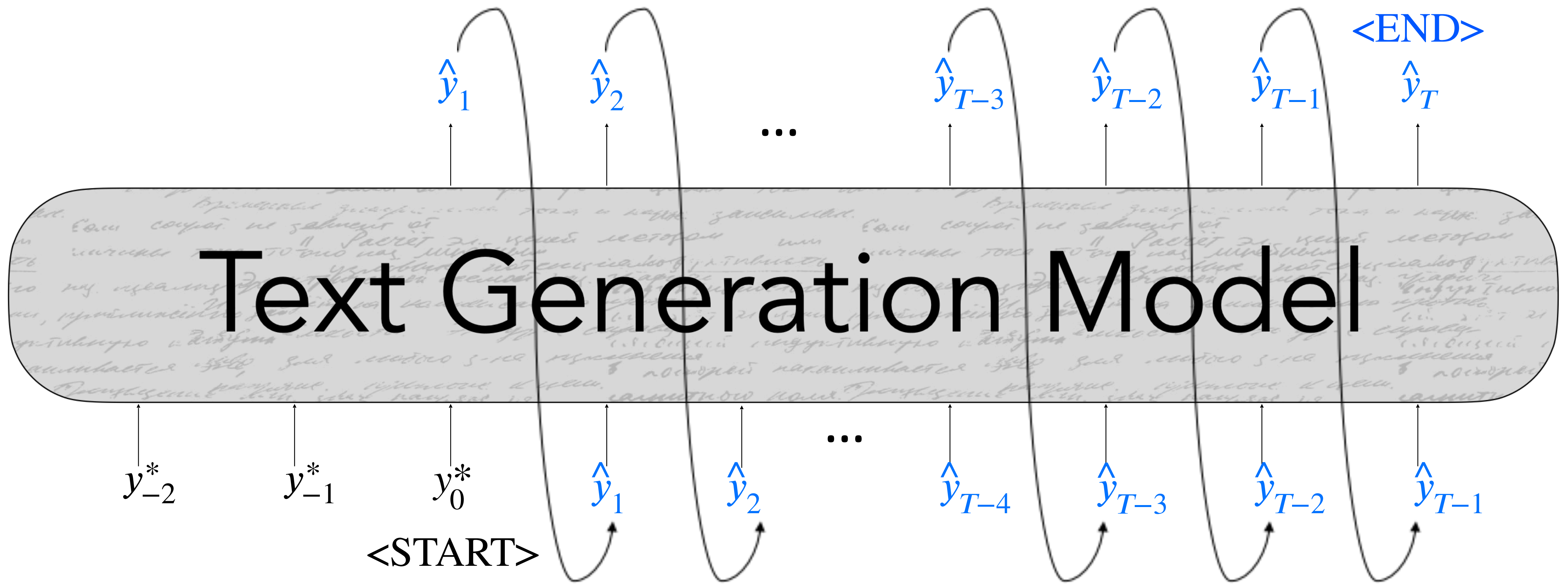
Solutions

- **Reinforcement Learning**: cast your text generation model as a Markov decision process
 - **State** s is the model's representation of the preceding context
 - **Actions** a are the words that can be generated
 - **Policy** π is the decoder
 - **Rewards** r are provided by an external score
 - Learn behaviors by rewarding the model when it exhibits them

REINFORCE: Basics

- Sample a sequence from your model

$$\mathcal{L}_{RL} = - \sum_{t=1}^T r(\hat{y}) \log P(\hat{y}_t | \{y^*\}; \{\hat{y}\}_{<t})$$



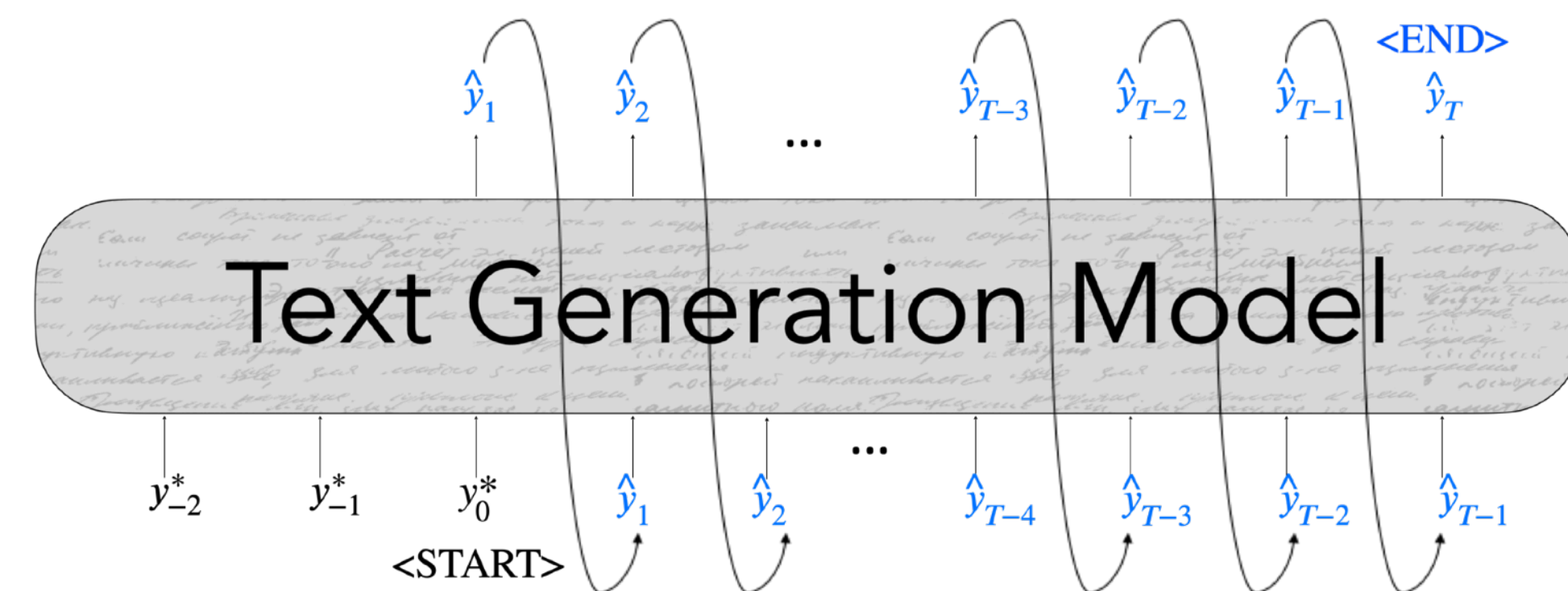
REINFORCE: Basics

- Sample a sequence from your model

Next time, increase the probability of this sampled token in the same context.

$$\mathcal{L}_{RL} = - \sum_{t=1}^T \log P(\hat{y}_t | \{y^*\}; \{\hat{y}\}_{<t})$$

...but do it more if I get a high reward from the reward function.

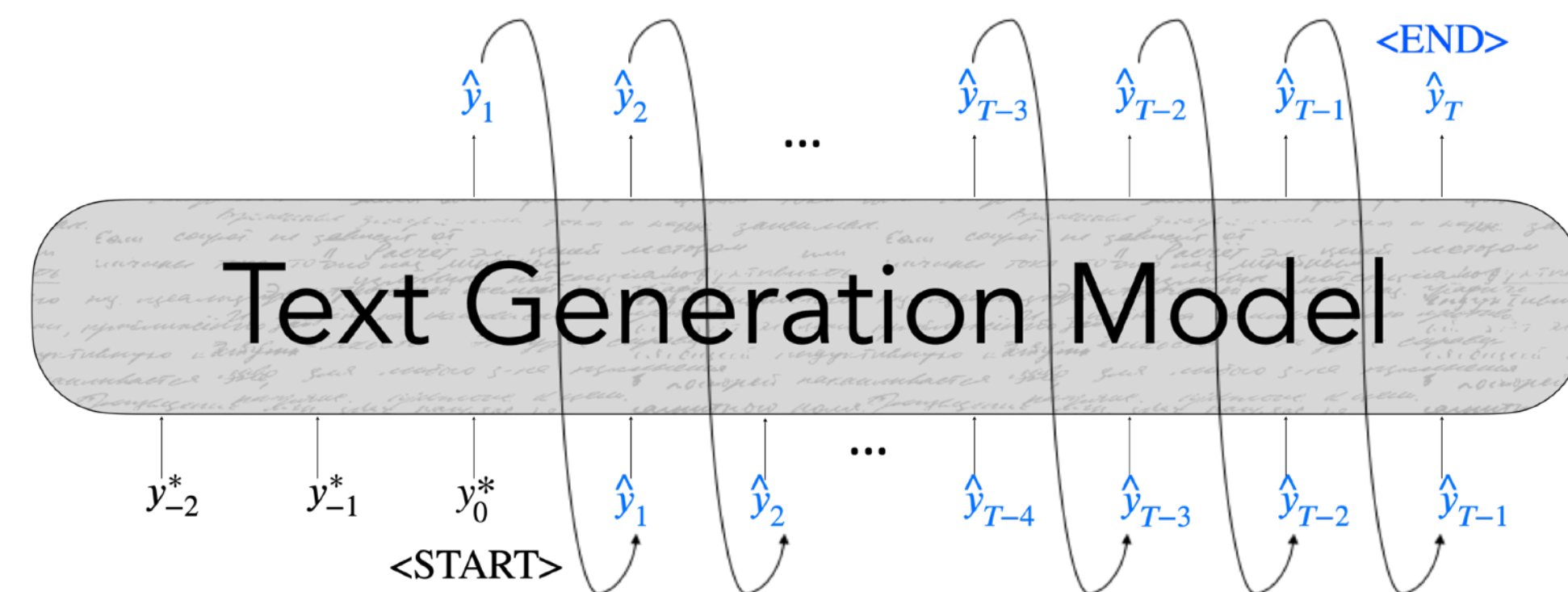


REINFORCE: Basics

- Sample a sequence from your model

$$\mathcal{L}_{RL} = -R(\hat{Y}) \sum_{t=1}^T \log P(\hat{y}_t | \{y^*\}; \{\hat{y}\}_{<t})$$

Often use an aggregate reward for the sequence — every token gets same reward



Reward Estimation

- How should we define a reward function? Just use your evaluation metric!
 - **BLEU** (machine translation; Ranzato et al., ICLR 2016; Wu et al., 2016)
 - **ROUGE** (summarization; Paulus et al., ICLR 2018; Celikyilmaz et al., NAACL 2018)
 - CIDEr (image captioning; Rennie et al., CVPR 2017)
 - SPIDEr (image captioning; Liu et al., ICCV 2017)

Reward Estimation

- How should we define a reward function? Just use your evaluation metric!
 - **BLEU** (machine translation; Ranzato et al., ICLR 2016; Wu et al., 2016)
 - **ROUGE** (summarization; Paulus et al., ICLR 2018; Celikyilmaz et al., NAACL 2018)
 - CIDEr (image captioning; Rennie et al., CVPR 2017)
 - SPIDEr (image captioning; Liu et al., ICCV 2017)
- Be careful about **optimizing for the task** as opposed to **“gaming” the reward!**
 - Evaluation metrics are merely proxies for generation quality!
 - “even though RL refinement can achieve better BLEU scores, it barely improves the human impression of the translation quality” – Wu et al., 2016

Reward Estimation

- What behaviors can we tie to rewards?
 - Cross-modality consistency in image captioning (Ren et al., CVPR 2017)
 - Sentence simplicity (Zhang and Lapata, EMNLP 2017)
 - Temporal Consistency (Bosselut et al., NAACL 2018)
 - Utterance Politeness (Tan et al., TACL 2018)
 - Paraphrasing (Li et al., EMNLP 2018)
 - Sentiment (Gong et al., NAACL 2019)
 - Formality (Gong et al., NAACL 2019)
- If you can formalize a behavior as a reward function (or train a neural network to approximate it!), you can train a text generation model to exhibit that behavior!
- Next week, we'll talk about **Reward Models** in more detail !

Implementation Thoughts

- Credit Assignment

$$r(\hat{y}_t) \text{ vs. } R(\hat{Y})$$

Implementation Thoughts

- Credit Assignment

$$r(\hat{y}_t) \text{ vs. } R(\hat{Y})$$

- Stability through variance reduction

$$\mathcal{L}_{RL} = - \sum (r(\hat{y}_t) - b) \log P(\hat{y}_t | \{\hat{y}\}_{<t}; \{y^*\})$$

Implementation Thoughts

- Credit Assignment

$$r(\hat{y}_t) \text{ vs. } R(\hat{Y})$$

- Stability through variance reduction

$$\mathcal{L}_{RL} = - \sum (r(\hat{y}_t) - b) \log P(\hat{y}_t | \{\hat{y}\}_{<t}; \{y^*\})$$

- Stability through joint optimisation

$$\mathcal{L} = \mathcal{L}_{MLE} + \alpha \mathcal{L}_{RL}$$

How could I use a “politeness classifier” as a reward function?

How might my model learn to exploit this?

Reward sequences that are classified as polite by the model.

Learn to add please 20x to the end of any utterance.

Training: Takeaways

- **Teacher forcing** is still the premier algorithm for training text generation models
- **Diversity** is an issue with sequences generated from teacher forced models
- **Exposure bias** causes text generation models to **lose coherence** easily
 - Models must learn to recover from their own bad samples (e.g., scheduled sampling, RL)
- Training with RL can also allow models to learn behaviors that are challenging to formalize
 - Learning can be **unstable**!

Decoding References

- [1] Gulcehre et al., On Using Monolingual Corpora in Neural Machine Translation. arXiv 2015
- [2] Wu et al., Google's Neural Machine Translation System: Bridging the Gap between Human and Machine Translation. arxiv 2016
- [3] Venugopalan et al., Improving LSTM-based Video Description with Linguistic Knowledge Mined from Text. EMNLP 2016
- [4] Li et al., A Diversity-Promoting Objective Function for Neural Conversation Models. EMNLP 2018
- [5] Paulus et al., A Deep Reinforced Model for Abstractive Summarization. ICLR 2018
- [6] Celikyilmaz et al., Deep Communicating Agents for Abstractive Summarization. NAACL 2018
- [7] Holtzman et al., Learning to Write with Cooperative Discriminators. ACL 2018
- [8] Fan et al., Hierarchical Neural Story Generation. ACL 2018
- [9] Gabriel et al., Discourse Understanding and Factual Consistency in Abstractive Summarization. EACL 2021
- [10] Dathathri et al., Plug and Play Language Models: A Simple Approach to Controlled Text Generation. ICLR 2020
- [11] Holtzman et al., The Curious Case of Neural Text Degeneration. ICLR 2020
- [12] Khandelwal et al., Generalization through Memorization: Nearest Neighbor Language Models. ICLR 2020

Training References

- [1] Bengio et. al., Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks. NeurIPS 2015
- [2] Ranzato et al., Sequence Level Training with Recurrent Neural Networks. ICLR 2016
- [3] Wu et al., Google's Neural Machine Translation System: Bridging the Gap between Human and Machine Translation. 2016
- [4] Ren et al., Deep Reinforcement Learning-based Image Captioning with Embedding Reward. CVPR 2017
- [5] Rennie et al., Self-critical Sequence Training for Image Captioning. CVPR 2017
- [6] Liu et al., Improved Image Captioning via Policy Gradient Optimization of SPIDeR. ICCV 2017
- [7] Zhang and Lapata, Sentence Simplification with Deep Reinforcement Learning. EMNLP 2017
- [8] Paulus et al., A Deep Reinforced Model for Abstractive Summarization. ICLR 2018
- [9] Celikyilmaz et al., Deep Communicating Agents for Abstractive Summarization. NAACL 2018
- [10] Bosselut et al., Discourse-Aware Neural Rewards for Coherent Text Generation. NAACL 2018
- [11] Tan and Bansal, Polite Dialogue Generation Without Parallel Data. TACL 2018
- [12] Li et al., Paraphrase Generation with Deep Reinforcement Learning. EMNLP 2018
- [13] Holtzman et. al., The Curious Case of Neural Text Degeneration. ICLR 2020